

Foundations of Artificial Intelligence

Chapter 03: Problem Solving as Search

Roberto Sebastiani

DISI, Università di Trento, Italy – roberto.sebastiani@unitn.it

https://disi.unitn.it/rseba/DIDATTICA/fai_2026/

Teaching assistant: **Nicola Debole**, nicola.debole@unitn.it,

M.S. Course “Artificial Intelligence Systems”, academic year 2026-2027

Last update: Friday 18th September, 2026, 08:51

Copyright notice: Most examples and images displayed in the slides of this course are taken from [Russell & Norwig, “Artificial Intelligence, a Modern Approach”, 3rd and 4th ed., Pearson], including explicitly figures from the above-mentioned books, so that their copyright is detained by the authors. A few other material (text, figures, examples) is authored by (in alphabetical order): Pieter Abbeel, Bonnie J. Dorr, Anca Dragan, Dan Klein, Nikita Kitaev, Tom Lenaerts, Michela Milano, Dana Nau, Maria Simi, who detain its copyright.

These slides cannot be displayed in public without the permission of the author.

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Problem Solving as Search

One of the dominant approaches to AI problem solving:
formulate a problem/task as search in a state space.

Main Paradigm

- 1 **Goal formulation:** define the successful states
 - Ex: a set of states, a Boolean test function ...
- 2 **Problem formulation:**
 - define a representation for states
 - define legal actions and transition functions
- 3 **Search:** find a solution by means of a search process
 - solutions are sequences of actions
- 4 **Execution:** given the solution, perform the actions

Problem Solving as Search: Example

Example: Traveling in Romania

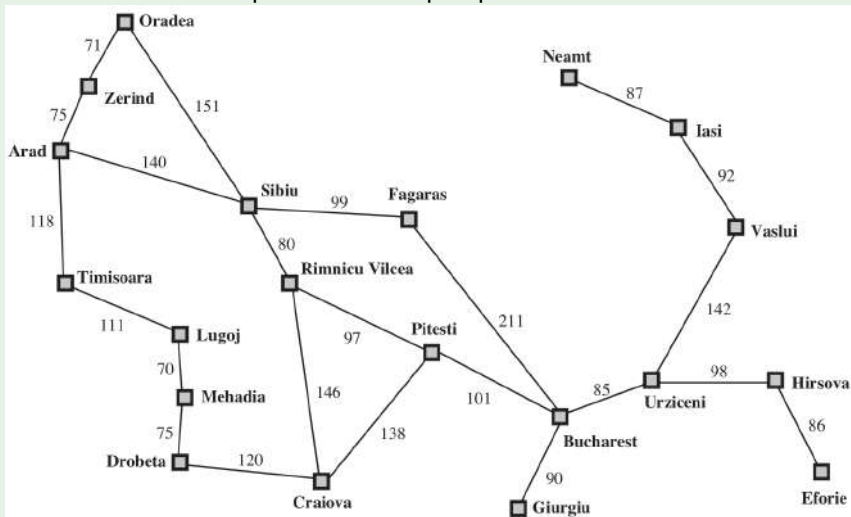
- **Informal description:**
On holiday in Romania; currently in Arad. Flight leaves tomorrow from Bucharest
- **Formulate goal:** (Be in) Bucharest
- **Formulate problem:**
 - States: various cities
 - Actions: drive between cities
 - Initial state: Arad
- **Search for a solution:** sequence of cities from Arad to Bucharest
 - e.g. Arad, Sibiu, Fagaras, Bucharest
 - explore a search tree/graph

Note

The agent is assumed to have no heuristic knowledge about traveling in Romania to exploit.

Problem Solving as Search: Example [cont.]

A simplified road map of part of Romania.



Problem Solving as Search: Example [cont.]

Assumptions (this example only)

- state representations are **atomic**
 - ⇒ world states are considered as wholes, with no internal structure
 - Ex: **Arad**, **Sibiu**, **Zerind**, **Bucharest**,... (shortcut for **In(Arad)**, **In(Sibiu)**, ...)
- the environment is **fully observable**
 - ⇒ the agent always knows the current state
 - Ex: **Romanian cities & roads have signs**
- the environment is **discrete**
 - ⇒ at any state there are only finitely many actions to choose from
 - Ex: from **Arad**, (go to) **Sibiu**, or **Zerind**, or **Timisoara** (see map)
- the environment is **known**
 - ⇒ the agent knows which states are reached by each action
 - ex: the agent has the map
- the environment is **deterministic**
 - ⇒ each action has exactly one outcome
 - Ex: from **Arad** choose **go to Sibiu** ⇒ next step in **Sibiu**

Problem Solving as Search [cont.]

Remark about state graphs

In practical search problems, we do not have a state graph!

- ⇒ **successor states must be computed on-the-fly by applying actions to current state**
- ⇒ do not confuse AI agent search problem with graph search problems (see examples later)

Remarks about search

- Search happens **inside the agent**
 - a **planning** stage before acting
 - different from **searching in the world**
- An agent is given **a description of what to achieve**, not an **algorithm to solve it**
 - ⇒ the only possibility is to **search for a solution**
- **Searching can be computationally very demanding (NP-hard)**
- **Can be driven with benefits by knowledge of the problem (heuristic knowledge)**
 - ⇒ **informed/heuristic search**

Well-defined problems and solutions

Problem Formulation: Components

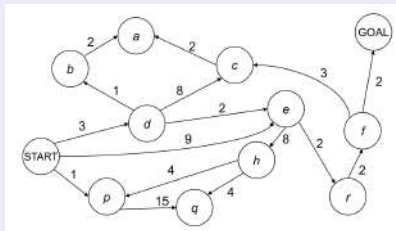
- The **initial state** the agent starts in
 - Ex: $In(Arad)$
- The set of **applicable actions** available in a state ($ACTIONS(s)$)
 - Ex: if s is $In(Arad)$, then the applicable actions are $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$
- A description of what each action does (aka **transition model**)
 - $RESULT(s,A)$: state resulting from applying action A in state s
 - Ex: $RESULT(IN(ARAD), GO(ZERIND))$ is $IN(ZERIND)$
- The **goal test** determining if a given state is a goal state
 - Explicit (e.g.: $\{In(Bucharest)\}$)
 - Implicit (e.g. (Ex: $CHECKMATE(x)$))
- The **path cost** function assigns a numeric cost ≥ 0 to each path
 - in this chapter: the sum of the costs of the actions along the path

Well-defined problems and solutions [cont.]

State Space, Graphs, Paths, Solutions and Optimal Solutions

Initial state, **actions**, and **transition model** implicitly define the **state space** of the problem

- the state space **implicitly** forms a **directed graph** (e.g. the Romania map)
 - typically too big and too hard to be created explicitly and be stored in full (e.g., $> 10^{100}$ for chess)
 - in a state space graph, each state occurs only once
- a **path** is a sequence of states connected by actions
- a **solution** is a path from the initial state to a goal state
- an **optimal solution** is a solution with the lowest path cost

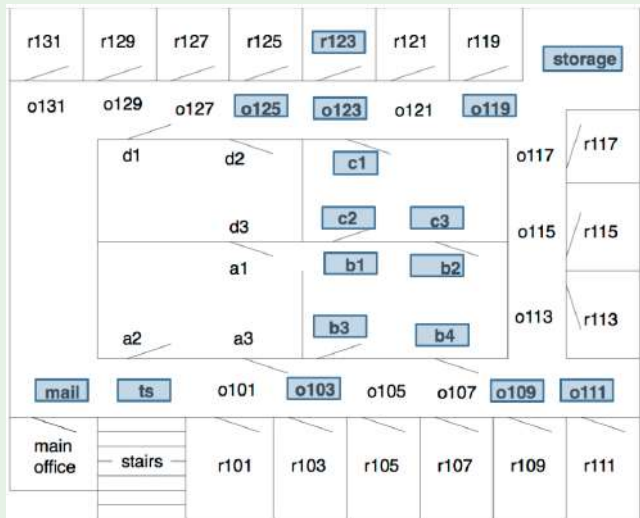


Example: Path finding for a Delivery Robot

Task: move from o103 to r123

- States

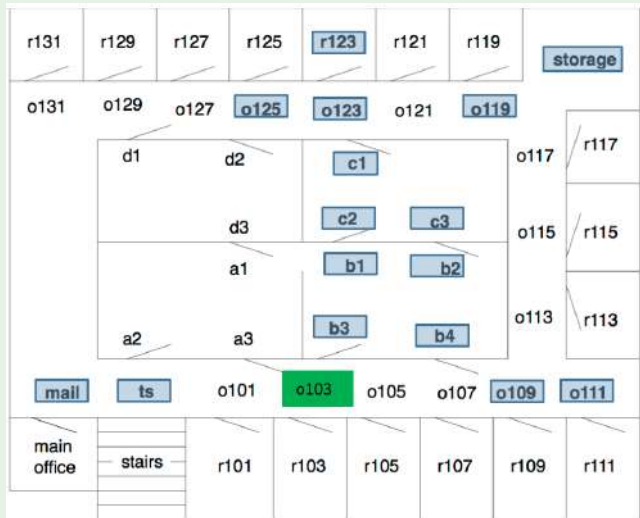
- Initial state
- Goal state
- State graph
- Optimal solution



Example: Path finding for a Delivery Robot

Task: move from o103 to r123

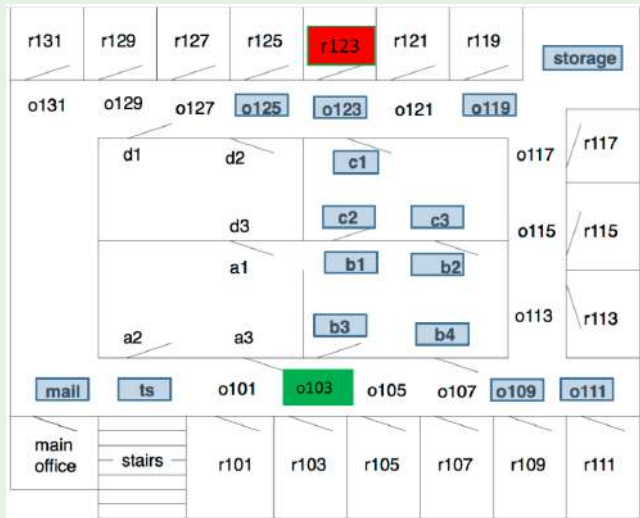
- States
- Initial state
- Goal state
- State graph
- Optimal solution



Example: Path finding for a Delivery Robot

Task: move from o103 to r123

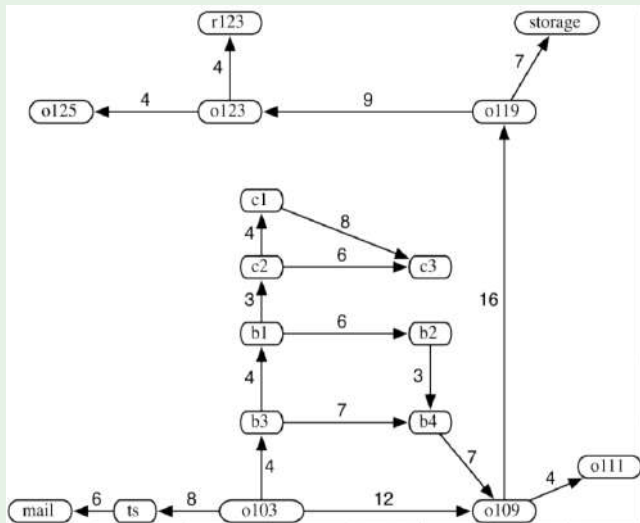
- States
- Initial state
- Goal state
- State graph
- Optimal solution



Example: Path finding for a Delivery Robot

Task: move from o103 to r123

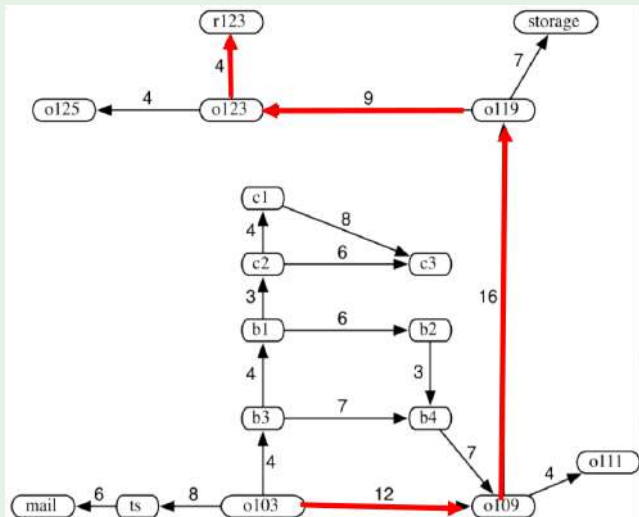
- States
- Initial state
- Goal state
- State graph
- Optimal solution



Example: Path finding for a Delivery Robot

Task: move from o103 to r123

- States
- Initial state
- Goal state
- State graph
- Optimal solution



Well-defined problems and solutions [cont.]

Abstraction

Problem formulations are **models** of reality (i.e. **abstract descriptions**)

- real world is absurdly complex
⇒ state space must be **abstracted** for problem solving
- lots of details removed because irrelevant to the problem
 - Ex: **exact position**, “**turn steering wheel to the left by 20 degree**”, ...
- **abstraction**: the process of removing detail from representations
 - **abstract state** represents many real states
 - **abstract action** represents complex combination of real actions
- **valid abstraction**: can expand any abstract solution into a solution in the detailed world
- **useful abstraction**: if carrying out each of the actions in the solution is easier than in the original problem

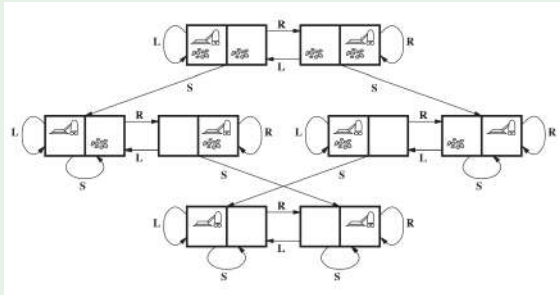
The **choice of a good abstraction** involves **removing as much detail as possible**, while **retaining validity** and **ensuring that the abstract actions are easy to carry out**.

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems**
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Toy Example: Simple Vacuum Cleaner

- **States:** 2 locations, each $\{clean, dirty\}$: $2 \cdot 2^2 = 8$ states
- **Initial State:** any
- **Actions:** $\{Left, Right, Suck\}$
- **Transition Model:** (...), Left [Right] if A [B], Suck if clean $\implies$ no effect
- **Goal Test:** check if squares are clean
- **Path Cost:** each step costs 1 $\implies$ path cost is # of steps in path



Toy Example: The 8-Puzzle

- **States:** Integer location of each tile $\Rightarrow 8!/2 = 20160$ reachable states
- **Initial State:** any
- **Actions:** moving $\{Left, Right, Up, Down\}$ the empty space
- **Transition Model:** empty space switched with the tile in target location
- **Goal Test:** checks state corresponds with goal configuration
- **Path Cost:** each step costs 1 $\Rightarrow$ path cost is # of steps in path

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

(© S. Russell & P. Norwig, AIMA)

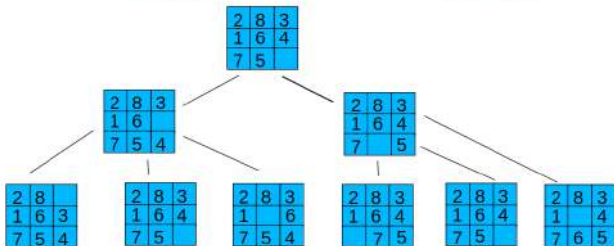
Toy Example: The 8-Puzzle [cont.]

Initial State

2	8	3
1	6	4
7	5	

Goal

1	2	3
8	4	
7	6	5



....

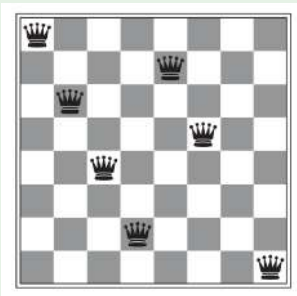
(Courtesy of Michela Milano, UniBO)

NP-complete: N-Puzzle ($N = k^2 - 1$): $N!/2$ reachable states

• Ex: $k = 3 \implies 20160$; $k = 4 \implies 653837184000$

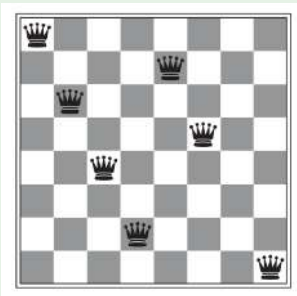
Toy Example: 8-Queens Problem

- **States:** any arrangement of 0 to 8 queens on the board
⇒ $64 \cdot 63 \cdot \dots \cdot 57 \approx 1.8 \cdot 10^{14}$ possible sequences
- **Initial State:** no queens on the board
- **Actions:** add a queen to any empty square
- **Transition Model:** returns the board with a queen added
- **Goal Test:** 8 queens on the board, none attacked by other queen
- **Path Cost:** none



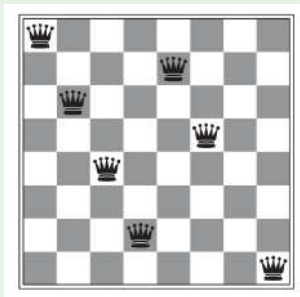
Toy Example: 8-Queens Problem

- **States:** any arrangement of 0 to 8 queens on the board
⇒ $64 \cdot 63 \cdot \dots \cdot 57 \approx 1.8 \cdot 10^{14}$ possible sequences
- **Initial State:** no queens on the board
- **Actions:** add a queen to any empty square
- **Transition Model:** returns the board with a queen added
- **Goal Test:** 8 queens on the board, none attacked by other queen
- **Path Cost:** none



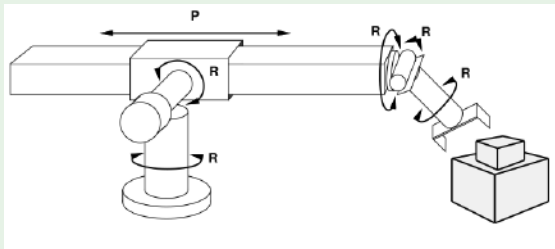
Toy Example: 8-Queens Problem (incremental)

- **States:** $n \leq 8$ queens on board, one per column in the n leftmost columns, **no queen attacking another**
⇒ 2057 possible sequences
- **Actions:** Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.
- ...



Real-World Example: Robotic Assembly

- **States:** real-valued coordinates of robot joint angles, and of parts of the object to be assembled
- **Initial State:** any arm position and object configuration
- **Actions:** continuous motions of robot joints
- **Transition Model:** position resulting from motion
- **Goal Test:** complete assembly (without robot)
- **Path Cost:** time to execute



Other Real-World Examples

- Airline travel planning problems
- Touring problems
- VLSI layout problem
- Robot navigation
- Automatic assembly sequencing
- Protein design
- ...

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities**
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Searching for Solutions

Search: Generate sequences of actions.

- **Expansion**: one starts from a state, and applying the operators (or successor function) will generate new states
- **Search strategy**: at each step, choose which state to expand.
- **Search Tree/DAG**: represents the expansion of all states starting from the initial state (the **root** of the tree/DAG)
- The **leaves** of the tree/DAG represent either:
 - states to expand
 - solutions
 - dead-ends

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities**
 - **Tree Search**
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Tree Search Algorithms

Tree Search: Basic idea

- Off-line, simulated exploration of state space
 - start from initial state
 - pick one leaf node, and generate its successors (a.k.a. **expanding** a node)
 - set of current leaves called **frontier** (a.k.a. **fringe**, **open list**)
 - strategy for picking leaves critical (**search strategy**)
 - ends when either **a goal state is reached**, or **no more candidates to expand are available** (or time-out/memory-out occur)

Tree Search Algorithms

Tree Search: Basic idea

- Off-line, simulated exploration of state space
 - start from initial state
 - pick one leaf node, and generate its successors (a.k.a. expanding a node)
 - set of current leaves called frontier (a.k.a. fringe, open list)
 - strategy for picking leaves critical (search strategy)
 - ends when either a goal state is reached, or no more candidates to expand are available (or time-out/memory-out occur)

Tree Search Algorithms

Tree Search: Basic idea

- Off-line, simulated exploration of state space
 - start from initial state
 - pick one leaf node, and generate its successors (a.k.a. **expanding** a node)
 - set of current leaves called **frontier** (a.k.a. **fringe**, **open list**)
 - strategy for picking leaves critical (**search strategy**)
 - ends when either a goal state is reached, or no more candidates to expand are available (or time-out/memory-out occur)

Tree Search Algorithms

Tree Search: Basic idea

- Off-line, simulated exploration of state space
 - start from initial state
 - pick one leaf node, and generate its successors (a.k.a. expanding a node)
 - set of current leaves called frontier (a.k.a. fringe, open list)
 - strategy for picking leaves critical (search strategy)
 - ends when either a goal state is reached, or no more candidates to expand are available (or time-out/memory-out occur)

Tree Search Algorithms

Tree Search: Basic idea

- Off-line, simulated exploration of state space
 - start from initial state
 - pick one leaf node, and generate its successors (a.k.a. **expanding** a node)
 - set of current leaves called **frontier** (a.k.a. **fringe**, **open list**)
 - strategy for picking leaves critical (**search strategy**)
 - ends when either **a goal state is reached**, or **no more candidates to expand are available** (or time-out/memory-out occur)

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

(© S. Russell & P. Norwig, AIMA)

Tree Search Algorithms

Tree Search: Basic idea

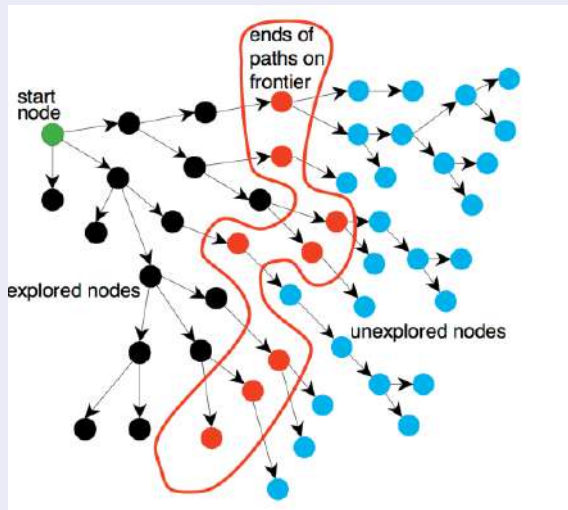
- Off-line, simulated exploration of state space
 - start from initial state
 - pick one leaf node, and generate its successors (a.k.a. **expanding** a node)
 - set of current leaves called **frontier** (a.k.a. **fringe**, **open list**)
 - strategy for picking leaves critical (**search strategy**)
 - ends when either **a goal state is reached**, or **no more candidates to expand are available** (or time-out/memory-out occur)

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

(© S. Russell & P. Norwig, AIMA)

The goal test on a node may be performed either before inserting it into the frontier (early test) or when extracting it from the frontier (late test)

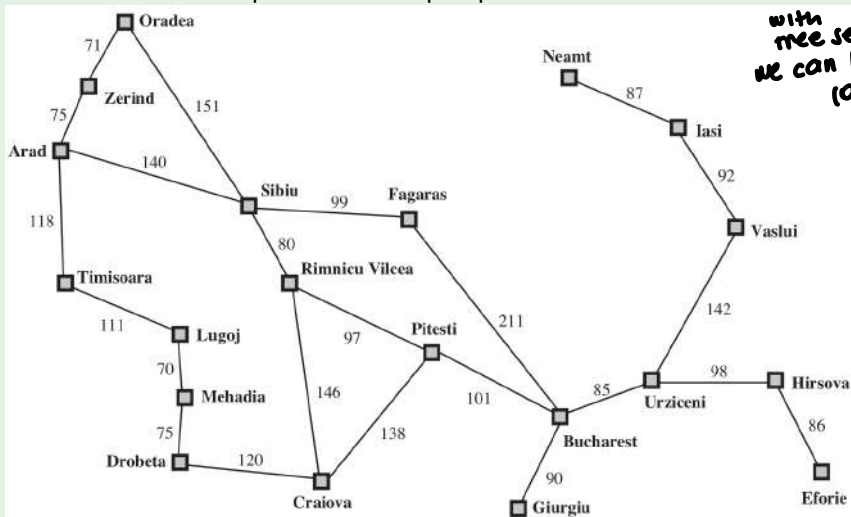
Tree Search Algorithms [cont.]



(Courtesy of Maria Simi, UniPI)

Tree-Search Example: Trip from Arad to Bucharest

A simplified road map of part of Romania.

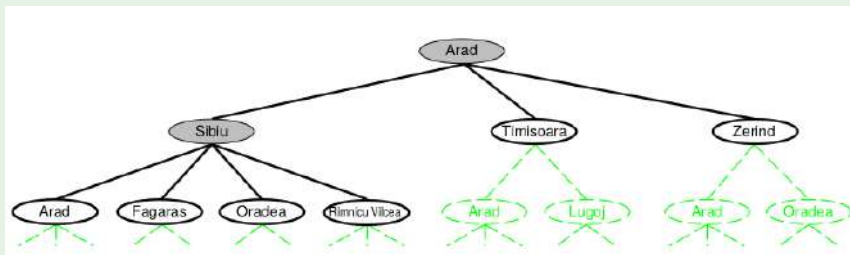


with
tree search
we can have
loops

Tree-Search Example: Trip from Arad to Bucharest

Expanding the search tree

- Initial state: {*Arad*}
- Expand initial state $\Rightarrow$ {*Sibiu*, *Timisoara*, *Zerind*}
- Pick&expand *Sibiu* $\Rightarrow$ {*Arad*, *Fagaras*, *Oradea*, *Rimnicu Vicea*}
- ...



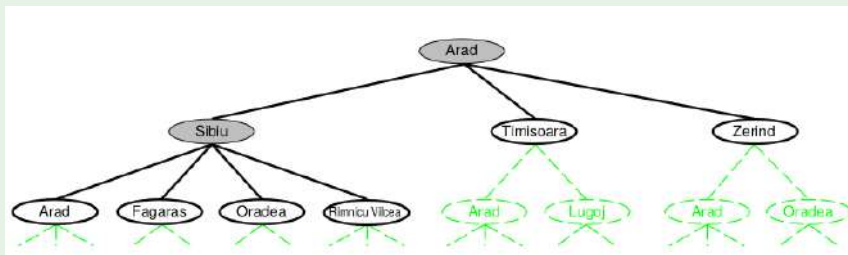
(© S. Russell & P. Norwig, AIMA)

Beware: $\text{Arad} \mapsto \text{Sibiu} \mapsto \text{Arad}$ (repeated state $\Rightarrow$ **loopy path!**)

Tree-Search Example: Trip from Arad to Bucharest

Expanding the search tree

- Initial state: {*Arad*}
- Expand initial state $\Rightarrow$ {*Sibiu*, *Timisoara*, *Zerind*}
- Pick&expand *Sibiu* $\Rightarrow$ {*Arad*, *Fagaras*, *Oradea*, *Rimnicu Vicea*}
- ...

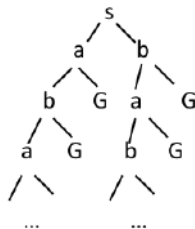
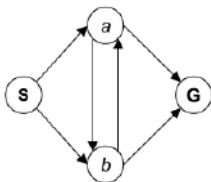


(© S. Russell & P. Norwig, AIMA)

Beware: *Arad* $\mapsto$ *Sibiu* $\mapsto$ *Arad* (repeated state $\Rightarrow$ **loopy path!**)

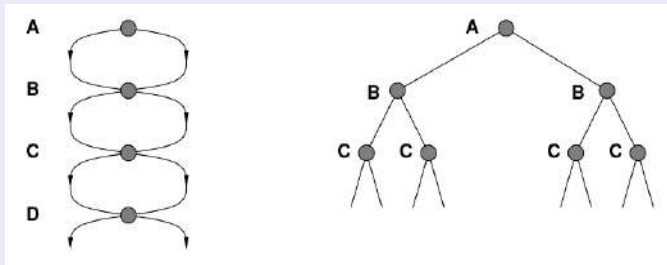
Repeated states & Redundant Paths

- **Redundant paths** occur when there is more than one way to get from one state to another
⇒ same state & subtree explored more than once
ex: *Arad, Sibiu* (*subtree*) vs. *Arad, Zerind, Oradea, Sibiu* (*same subtree*)
- Failure to detect repeated states can:
 - **cause infinite loops**
 - **turn linear problem into exponential**



Repeated states & Redundant Paths

- **Redundant paths** occur when there is more than one way to get from one state to another
⇒ same state & subtree explored more than once
ex: *Arad, Sibiu* (*subtree*) vs. *Arad, Zerind, Oradea, Sibiu* (*same subtree*)
- Failure to detect repeated states can:
 - cause infinite loops
 - turn linear problem into exponential

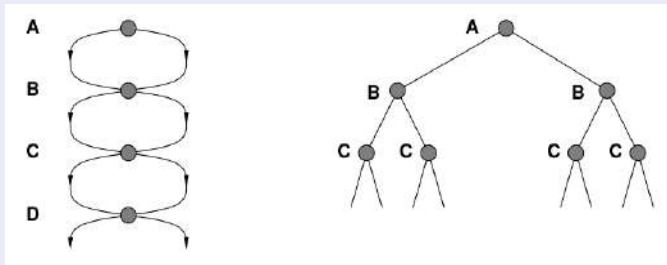


(© S. Russell & P. Norwig, AIMA)

Moral: Algorithms that forget their history are doomed to repeat it!

Repeated states & Redundant Paths

- **Redundant paths** occur when there is more than one way to get from one state to another
⇒ same state & subtree explored more than once
ex: *Arad, Sibiu* (*subtree*) vs. *Arad, Zerind, Oradea, Sibiu* (*same subtree*)
- Failure to detect repeated states can:
 - cause infinite loops
 - turn linear problem into exponential



(© S. Russell & P. Norwig, AIMA)

Moral: Algorithms that forget their history are doomed to repeat it!

Outline

- 1 Problem-Solving Agents
 - 2 Some Example Problems
 - 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
 - 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
 - 5 Informed Search Strategies
 - Greedy Best-First Search
 - A* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions
- never harra aca*

Graph Search Algorithms

Graph Search: Basic idea

- Add a data structure which remembers every expanded node
 - a.k.a. explored set or closed list
 - typically a hash table (access $O(1)$)
- Do not expand a node if it already occurs in explored set

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure

initialize the frontier using the initial state of *problem*

initialize the explored set to be empty

loop do

if the frontier is empty **then return** failure

choose a leaf node and remove it from the frontier

if the node contains a goal state **then return** the corresponding solution late test

add the node to the explored set

expand the chosen node, adding the resulting nodes to the frontier

only if not in the frontier or explored set

Graph Search Algorithms

Graph Search: Basic idea

- Add a data structure which remembers every expanded node
 - a.k.a. explored set or closed list
 - typically a hash table (access $O(1)$)
- Do not expand a node if it already occurs in explored set

```
function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set
```

Graph Search Algorithms

Graph Search: Basic idea

- Add a data structure which remembers every expanded node
 - a.k.a. explored set or closed list
 - typically a hash table (access $O(1)$)
- Do not expand a node if it already occurs in explored set

Sometimes you can afford to do the early tests depending on the goal

```
function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set
```

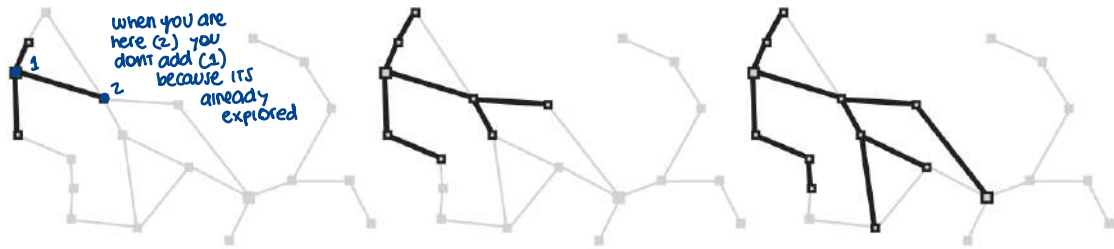
(© S. Russell & P. Norwig, AIMA)

The goal test on a node may be performed either before inserting it into the frontier (early test) or when extracting it from the frontier (late test)

Graph Search Algorithms: Example

Graph search on the Romania trip problem

- (at each stage each path extended by one step)
- two states become dead-end



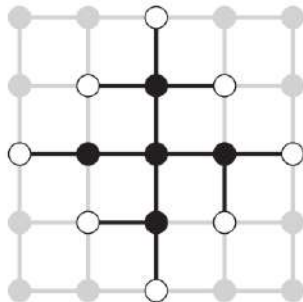
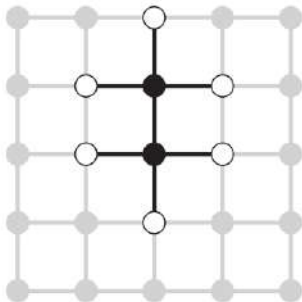
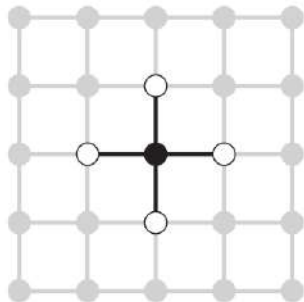
(© S. Russell & P. Norwig, AIMA)

Graph Search Algorithms: Example

Separation Property of graph search:

The frontier separates the state-space graph into the **explored region** and the **unexplored region**

Ex: Graph search on a rectangular-grid problem



(© S. Russell & P. Norwig, AIMA)

Intermediate scenarios between tree and graph search

Tree vs. graph search

- **Tree:** search less efficient (repeated search, loops) *you don't keep memory (no consumption)*
- **Graph:** high memory consumption (up to exponential) *can be*

Remark: loop detection

- **Loop detection** in tree search is relatively easy:
adding a new node, follow up the chain of parent pointers (see later)

⇒ avoid loops

⇒ does not avoid redundant subtrees

Personal remark (not on AIMA book)

- “Explored” can be implemented as **bounded-size cache** of nodes
 - periodically remove nodes with least cache hits
 - tradeoff between search efficiency and memory size
- ⇒ **intermediate between tree and graph search:**
the larger the cache, the more “graph-like” the search

Intermediate scenarios between tree and graph search

Tree vs. graph search

- **Tree:** search less efficient (repeated search, loops)
- **Graph:** high memory consumption (up to exponential)

Remark: loop detection

- **Loop detection** in tree search is relatively easy:
adding a new node, follow up the chain of parent pointers (see later)

⇒ avoid loops

⇒ does not avoid redundant subtrees

Personal remark (not on AIMA book)

- “Explored” can be implemented as **bounded-size cache** of nodes
 - periodically remove nodes with least cache hits
 - tradeoff between search efficiency and memory size
- ⇒ **intermediate between tree and graph search:**
the larger the cache, the more “graph-like” the search

Intermediate scenarios between tree and graph search

Tree vs. graph search

- **Tree:** search less efficient (repeated search, loops)
- **Graph:** high memory consumption (up to exponential)

Remark: loop detection

- **Loop detection** in tree search is relatively easy:
adding a new node, follow up the chain of parent pointers (see later)

⇒ avoid loops

⇒ does not avoid redundant subtrees

Personal remark (not on AIMA book)

- “Explored” can be implemented as **bounded-size cache** of nodes
 - periodically remove nodes with least cache hits
 - tradeoff between search efficiency and memory size

⇒ **intermediate between tree and graph search:**
the larger the cache, the more “graph-like” the search

*IMPORTANT:
there are intermediate
scenarios*

Outline

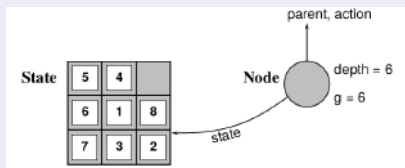
- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities**
 - Tree Search
 - Graph Search
 - **Implementation Issues & Strategies**
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Implementation: States vs. Nodes

- A **state** is a representation of a physical configuration
- A **node** is a data structure constituting part of a search tree
 - includes fields: **state**, **parent**, **action**, **path cost $g(x)$**

⇒ **node** $\neq$ **state**

- Within a given problem, it should be easy to compute a child node from its parent and the action performed



```
function CHILD-NODE(problem, parent, action) returns a node
return a node with
    STATE = problem.RESULT(parent.STATE, action),
    PARENT = parent, ACTION = action,
    PATH-COST = parent.PATH-COST + problem.STEP-COST(parent.STATE, action)
```

Implementation: Frontier and Explored

Frontier/Fringe

- Implemented as a Queue:
 - First-in-First-Out, FIFO (aka “queue”): $O(1)$ access *constant time*
 - Last-in-First-Out, LIFO (aka “stack”): $O(1)$ access
 - Best-First-out (aka “priority queue”): $O(\log(n))$ access
- Three primitives:
 - `ISEMPTY(Queue)`: returns true iff there are no more elements
 - `POP(Queue)`: removes and returns the first element of the queue
 - push* → ● `INSERT(ELEMENT, Queue)`: inserts an element into queue

Explored

- Implemented as a Hash Table: $O(1)$ access
- Two primitives:
 - `ISTHERE(ELEMENT, HASH)`: returns true iff element is in the hash
 - `INSERT(ELEMENT, HASH)`: inserts element into hash
- Choice of hash function critical for efficiency

Uninformed vs. Informed Search Strategies

Strategies: Two possibilities

- **Uninformed** strategies (a.k.a. **blind** strategies)
 - do not use any domain knowledge
 - apply rules arbitrarily and do an exhaustive search strategy
 - ⇒ impractical for some complex problems.
- **Informed** strategies
 - use domain knowledge
 - apply rules following heuristics (driven by domain knowledge)
 - ⇒ practical for many complex problems.

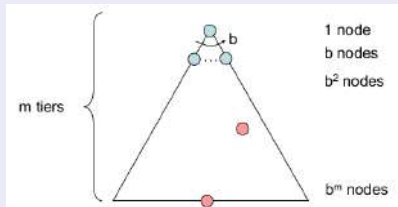
Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies**
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Evaluating Search Strategies

- Strategies are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: how many steps to find a solution?
 - **space complexity**: how much memory is needed?
 - **optimality**: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - **b**: maximum branching factor of the search tree
 - **d**: depth of the least-cost solution
 - **m**: maximum depth of the state space (may be $+\infty$)

⇒ # nodes: $1 + b + b^2 + \dots + b^m = O(b^m)$



Uninformed Search Strategies

Uninformed strategies

Use only the information available in the problem definition

- Different uninformed search strategies
 - Best-first/Uniform-cost search
 - Breadth-first search
 - Depth-first search
 - Depth-limited search & Iterative-deepening search
- Defined by **the access strategy of the frontier/fringe** (i.e. the order of node expansion)
 - **goal test strategy** may vary as well

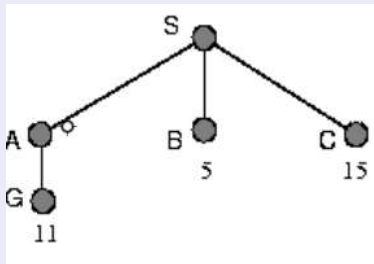
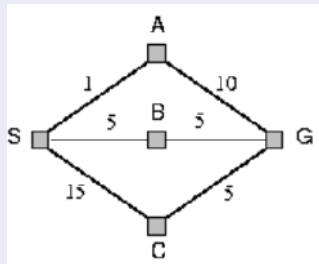
Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies**
 - Best-first/Uniform-cost Search**
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Best-First / Uniform-Cost Search Strategy (UCS)

Best-first search (aka Uniform-Cost Search)

- Idea: **Expand first the node with lowest path cost $g(n)$**
- Implementation: **frontier/fringe implemented as a priority queue ordered by $g()$**
⇒ novel nearest successors pushed to the top of the queue
- goal test tricky (see next slide)

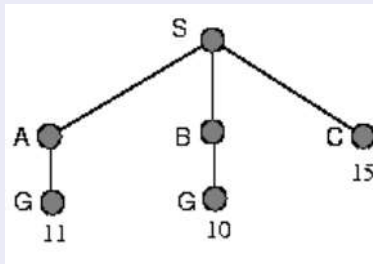
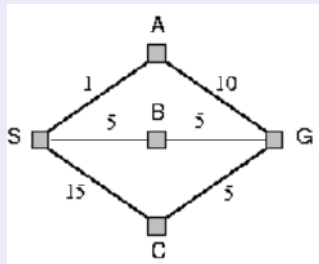


(Courtesy of Michela Milano, UniBO)

Best-First / Uniform-Cost Search Strategy (UCS)

Best-first search (aka Uniform-Cost Search)

- Idea: **Expand first the node with lowest path cost $g(n)$**
- Implementation: **frontier/fringe implemented as a priority queue ordered by $g()$**
⇒ novel nearest successors pushed to the top of the queue
- goal test tricky (see next slide)



late test: when we start from the frontier

(Courtesy of Michela Milano, UniBO)

Uniform-Cost (aka Best-first) Search Strategy (UCS) [cont.]

Graph version (Tree version: without “reached”)

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution node, or *failure*
return BEST-FIRST-SEARCH(*problem*, PATH-COST)

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*
node $\leftarrow$ NODE(STATE=*problem*.INITIAL)
frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element
reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*
while not IS-EMPTY(*frontier*) **do**
 node $\leftarrow$ POP(*frontier*)
 if *problem*.IS-GOAL(*node*.STATE) **then return** *node* // late test
 for each *child* **in** EXPAND(*problem*, *node*) **do**
 s $\leftarrow$ *child*.STATE // re-added if now reached by another path with lower cost
 if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**
 reached[*s*] $\leftarrow$ *child* // If Tree search, just “add child to frontier” directly, without if test
 add *child* to *frontier*
return failure

loop
while
frontier
not empty

function EXPAND(*problem*, *node*) **yields** nodes // generates a sequence of nodes
s $\leftarrow$ *node*.STATE
for each *action* **in** *problem*.ACTIONS(*s*) **do**
 s' $\leftarrow$ *problem*.RESULT(*s*, *action*)
 cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)
 yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)
 // add a node to the sequence

Uniform-Cost (aka Best-first) Search Strategy (UCS) [cont.]

- apply the goal test to a node **when it is selected from the frontier for expansion** (late test), rather than **when it is first generated, before being added to the frontier**
- re-add in the frontier a node with same state but worse path cost

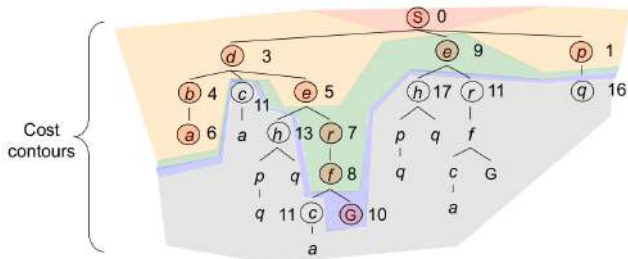
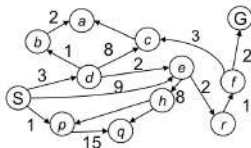
⇒ **avoid generating suboptimal paths** (see previous example)

Uniform-Cost Search

UCS (Tree version)

Strategy: expand a
cheapest node first:

Fringe is a priority queue
(priority: cumulative cost)



node	priority queue
$S_{[0]}$	$\{S_{[0]}\}$
$p_{[1]}$	$\{p_{[1]}, d_{[3]}, e_{[9]}\}$
$d_{[3]}$	$\{d_{[3]}, e_{[9]}, q_{[16]}\}$
$b_{[4]}$	$\{e_{[5]}, a_{[6]}, e_{[9]}, c_{[11]}, q_{[16]}\}$
$e_{[5]}$	$\{a_{[6]}, r_{[7]}, e_{[9]}, c_{[11]}, h_{[13]}, q_{[16]}\}$
$a_{[6]}$	$\{r_{[7]}, e_{[9]}, c_{[11]}, h_{[13]}, q_{[16]}\}$
$r_{[7]}$	$\{e_{[9]}, e_{[9]}, c_{[11]}, h_{[13]}, q_{[16]}\}$
$f_{[8]}$	$\{e_{[9]}, G_{[10]}, c_{[11]}, c_{[11]}, h_{[13]}, q_{[16]}\}$
$e_{[9]}$	$\{G_{[10]}, c_{[11]}, c_{[11]}, r_{[11]}, h_{[13]}, q_{[16]}, h_{[17]}\}$
$G_{[10]}$	Goal reached (optimumpath)!

Exercises

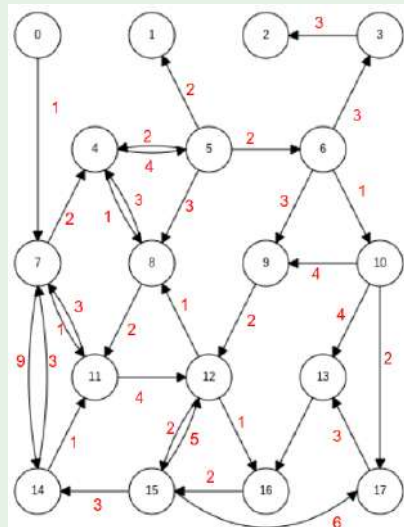
- ➊ Apply UCS, graph version, to previous example
- ➋ Apply UCS to the Romania-map Example
(both tree and graph versions)
- ➌ Consider the following graph, initial state 0, goal state 17
 - ➊ explore it using UCS (tree version)
 - ➋ explore it using UCS (graph version)

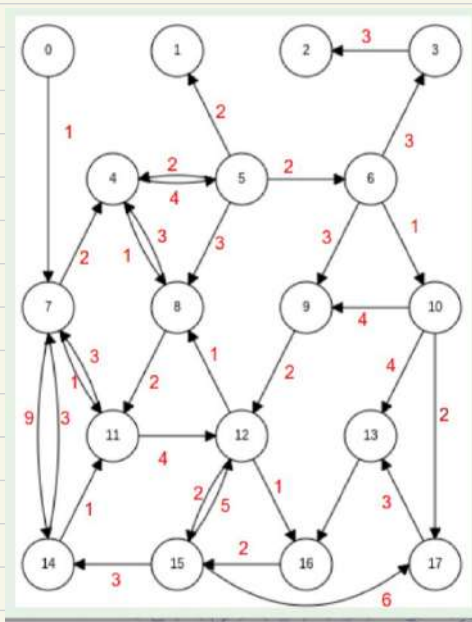
Exercises

- 1 Apply UCS, graph version, to previous example
- 2 Apply UCS to the Romania-map Example
(both tree and graph versions)
- 3 Consider the following graph, initial state 0, goal state 17
 - explore it using UCS (tree version)
 - explore it using UCS (graph version)

Exercises

- ➊ Apply UCS, graph version, to previous example
- ➋ Apply UCS to the Romania-map Example (both tree and graph versions)
- ➌ Consider the following graph, initial state 0, goal state 17
 - ➊ explore it using UCS (tree version)
 - ➋ explore it using UCS (graph version)





node

priority queue

$\{0_{[0]}\}$

$0_{[0]}$

$\{7_{[1]}\}$

$7_{[1]}$

$\{11_{[2]}, 14_{[10]}\}$

Uniform-Cost Search (UCS): Properties

C^* : cost of cheapest solution; ϵ : minimum arc cost

$\Rightarrow 1 + \lfloor C^*/\epsilon \rfloor$ “effective depth”

- How many steps?

- processes all nodes costing less than cheapest solution

$\Rightarrow$ takes $O(b^{1+\lfloor C^*/\epsilon \rfloor})$ time

- How much memory?

- max frontier size: $b^{1+\lfloor C^*/\epsilon \rfloor}$

$\Rightarrow O(b^{1+\lfloor C^*/\epsilon \rfloor})$ memory size

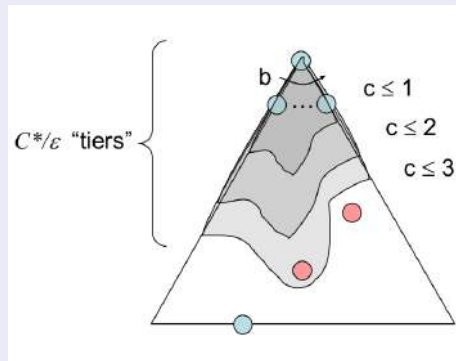
- Is it complete?

- if solution exists, finite cost

$\Rightarrow$ Yes

- Is it optimal?

- Yes



(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement is a major problem for uniform-cost search

Uniform-Cost Search (UCS): Properties

maximum integer
that is $> X$

C^* : cost of cheapest solution; ϵ : minimum arc cost

$\Rightarrow 1 + \lfloor C^*/\epsilon \rfloor$ “effective depth”

- How many steps?

- processes all nodes costing less than cheapest solution

$\Rightarrow$ takes $O(b^{1+\lfloor C^*/\epsilon \rfloor})$ time

- How much memory?

- max frontier size: $b^{1+\lfloor C^*/\epsilon \rfloor}$

$\Rightarrow O(b^{1+\lfloor C^*/\epsilon \rfloor})$ memory size

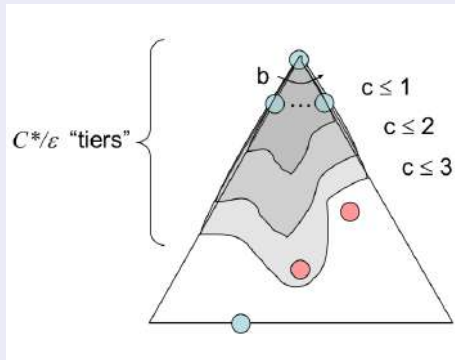
- Is it complete?

- if solution exists, finite cost

$\Rightarrow$ Yes

- Is it optimal?

- Yes



(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement is a major problem for uniform-cost search

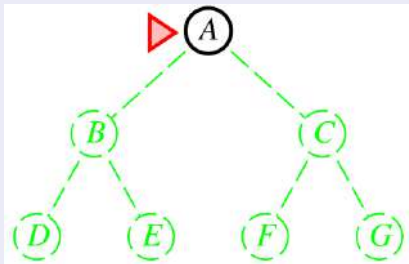
Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies**
 - Best-first/Uniform-cost Search
 - Breadth-First Search**
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Breadth-First Search Strategy (BFS)

Breadth-First Search *no path cost*

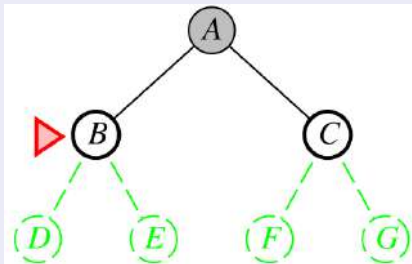
- Idea: **Expand first the shallowest unexpanded nodes**
- Implementation: **frontier/fringe implemented as a FIFO queue**
⇒ novel successors pushed to the end of the queue



Breadth-First Search Strategy (BFS)

Breadth-First Search

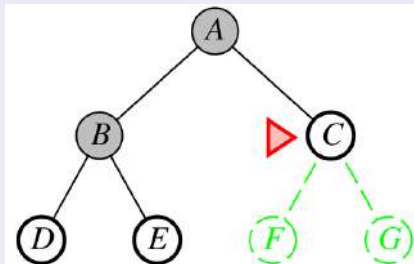
- Idea: Expand first the shallowest unexpanded nodes
- Implementation: frontier/fringe implemented as a FIFO queue
⇒ novel successors pushed to the end of the queue



Breadth-First Search Strategy (BFS)

Breadth-First Search

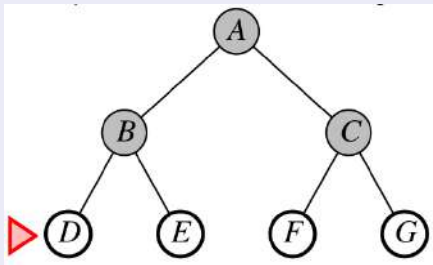
- Idea: Expand first the shallowest unexpanded nodes
- Implementation: frontier/fringe implemented as a FIFO queue
⇒ novel successors pushed to the end of the queue



Breadth-First Search Strategy (BFS)

Breadth-First Search

- Idea: Expand first the shallowest unexpanded nodes
- Implementation: frontier/fringe implemented as a FIFO queue
⇒ novel successors pushed to the end of the queue



Breadth-First Search Strategy (BFS) [cont.]

BFS, Graph version (Tree version without “reached”)

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node ← NODE(problem.INITIAL)    // early test  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier ← a FIFO queue, with node as an element  
  reached ← {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node ← POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s ← child.STATE  
      if problem.IS-GOAL(s) then return child    // early test  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```

(© S. Russell & P. Norwig, AIMA)

EXPAND(): same as with best-first-search

Note: the goal test is applied to each node **when it is generated, before being added to the frontier** (early test), rather than when **it is selected from the frontier for expansion**

⇒ solution detected 1 layer earlier

Remark

In principle, BFS could be implemented as UCS (“best first”) using $cost = 1$ for each action, but ...

- Access to FIFO queue ($O(1)$) faster than that to priority queue ($O(\log(n))$)
- Beware of goal test!
 - UCS performs “late test”: when extracting node from the frontier
 - BFS performs “early test”: when inserting node into the frontier

⇒ Implementing BFS as UCS would be much less efficient

Remark

In principle, BFS could be implemented as UCS (“best first”) using $cost = 1$ for each action, but ...

- Access to FIFO queue ($O(1)$) faster than that to priority queue ($O(\log(n))$)
- **Beware of goal test!**
 - UCS performs “late test”: when extracting node from the frontier
 - BFS performs “early test”: when inserting node into the frontier

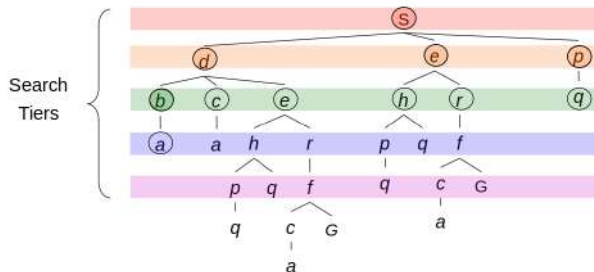
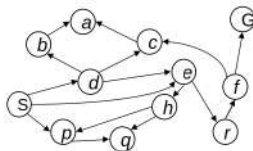
⇒ **Implementing BFS as UCS would be much less efficient**

Breadth-First Search: Tiers

State space explored by tiers (tree version, children expanded in alphabetical order)

Strategy: expand a shallowest node first

Implementation: Fringe is a FIFO queue



node	queue
	{S}
S	{d, e, p}
d	{e, p, b, c, e}
e	{p, b, c, e, h, r}
p	{b, c, e, h, r, q}
b	{c, e, h, r, q, a}
c	{e, h, r, q, a, a}
e	{h, r, q, a, a, h, r}
h	{r, q, a, a, h, r, p, q}
r	{q, a, a, h, r, p, q, f}
q	{a, a, h, r, p, q, f}
a	{a, h, r, p, q, f}
a	{h, r, p, q, f}
h	{r, p, q, f, p, q}
r	{p, q, f, p, q, f}
p	{q, f, p, q, f, q}
q	{f, p, q, f, q}
f	{p, q, f, q, c, G} Goal reached!

BFS Example

BFS Search for 8-Puzzle (tree version)

Expansion order: (move empty cell) up, left, right, down.



Frontier



Goal

BFS Example

BFS Search for 8-Puzzle (tree version)

Expansion order: (move empty cell) up, left, right, down.



Frontier



Goal

4	1	3
7	2	6
	5	8

BFS Example

BFS Search for 8-Puzzle (tree version)

Expansion order: (move empty cell) up, left, right, down.



Frontier



Goal

4	1	3
7	2	6
	5	8

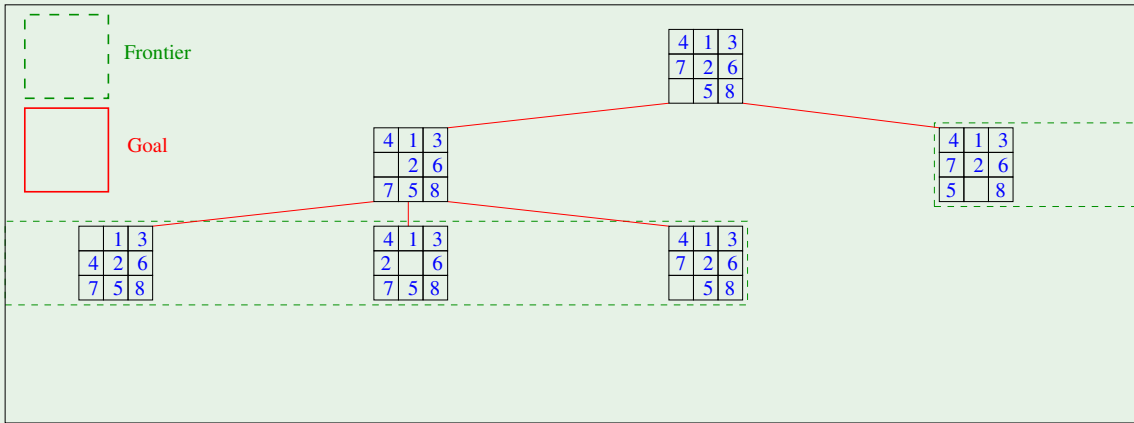
4	1	3
	2	6
7	5	8

4	1	3
7	2	6
5		8

BFS Example

BFS Search for 8-Puzzle (tree version)

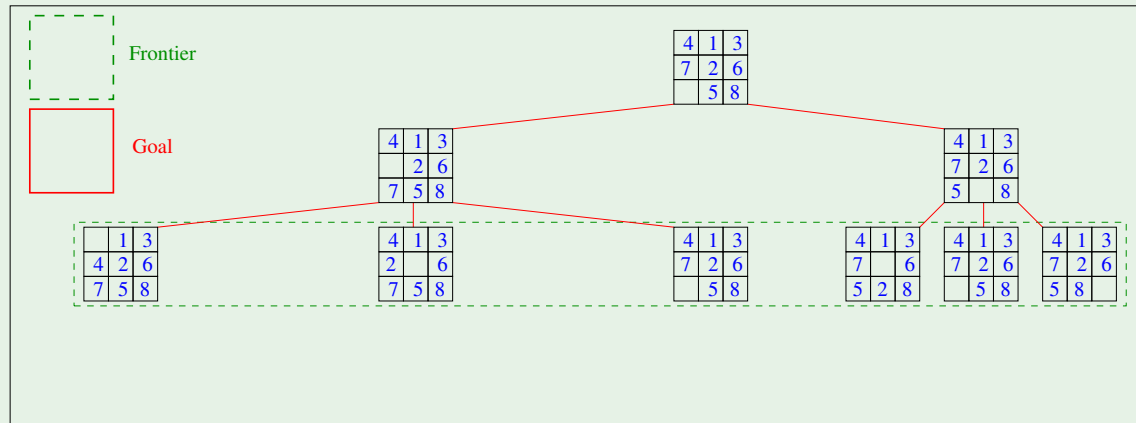
Expansion order: (move empty cell) up, left, right, down.



BFS Example

BFS Search for 8-Puzzle (tree version)

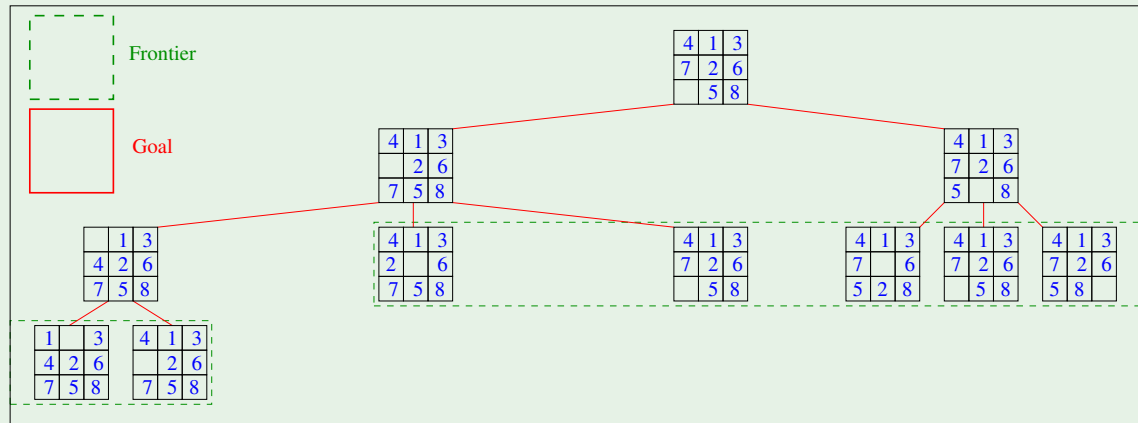
Expansion order: (move empty cell) up, left, right, down.



BFS Example

BFS Search for 8-Puzzle (tree version)

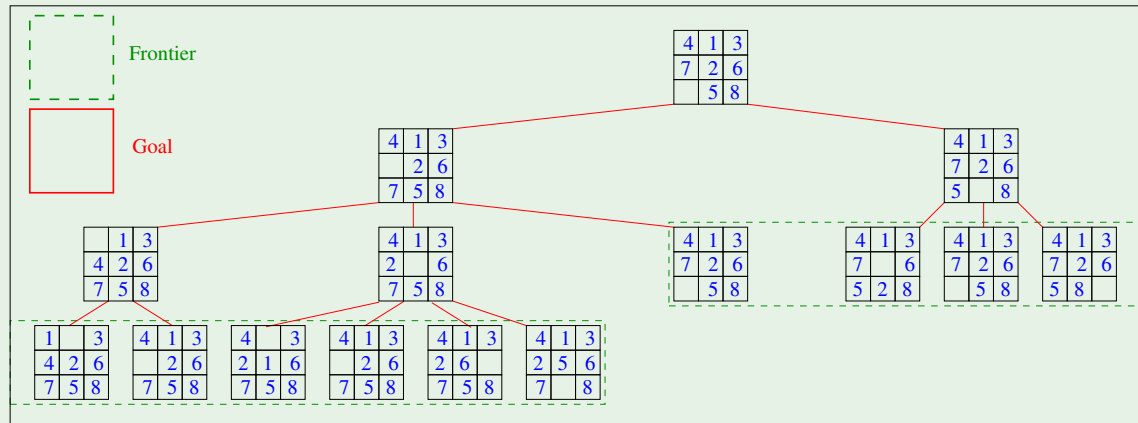
Expansion order: (move empty cell) up, left, right, down.



BFS Example

BFS Search for 8-Puzzle (tree version)

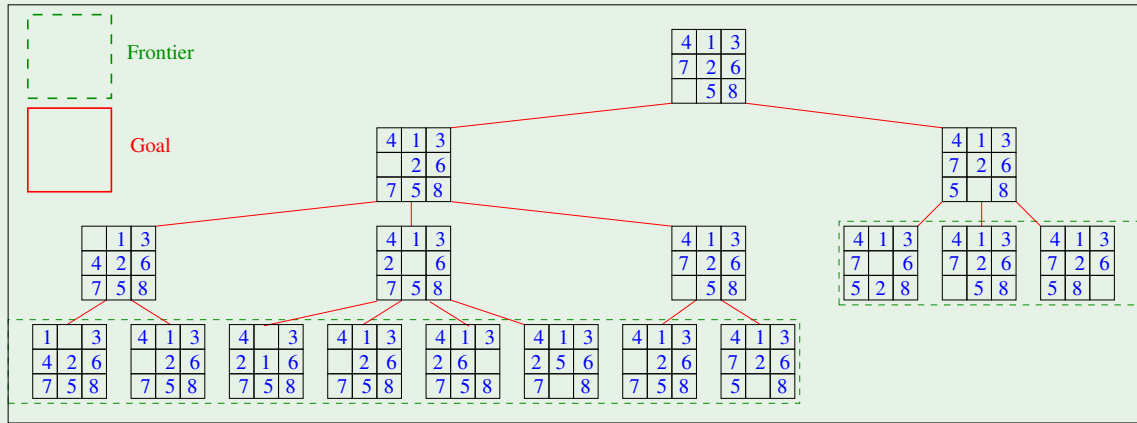
Expansion order: (move empty cell) up, left, right, down.



BFS Example

BFS Search for 8-Puzzle (tree version)

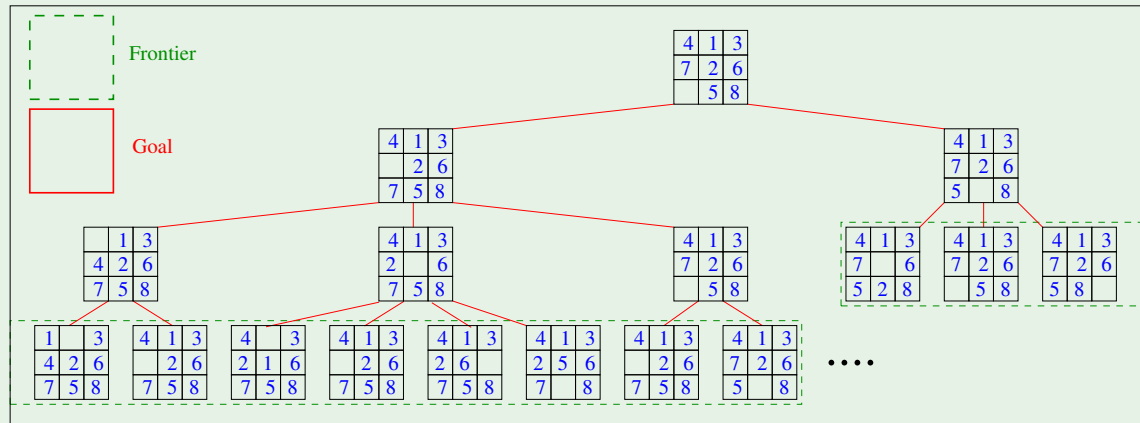
Expansion order: (move empty cell) up, left, right, down.



BFS Example

BFS Search for 8-Puzzle (tree version)

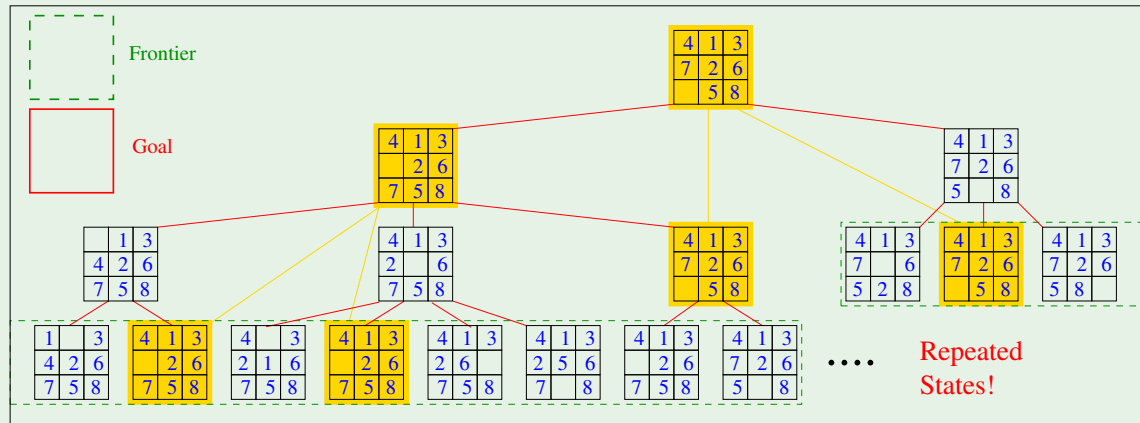
Expansion order: (move empty cell) up, left, right, down.



BFS Example

BFS Search for 8-Puzzle (tree version)

Expansion order: (move empty cell) up, left, right, down.

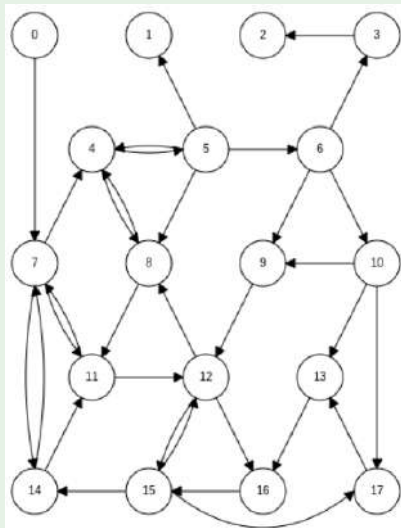


Exercises

- 1 Run previous examples, with BFS **graph** search.
- 2 Consider the following graph, initial state 0, goal state 17:
 - 1 explore it using BFS, tree version
 - 2 explore it using BFS, graph versionchildren should be expanded in numerical order
- 3 Solve the 4-queen problem (incremental version) by BFS
 - children should be expanded in row-index order
 - do tree search and graph search differ?

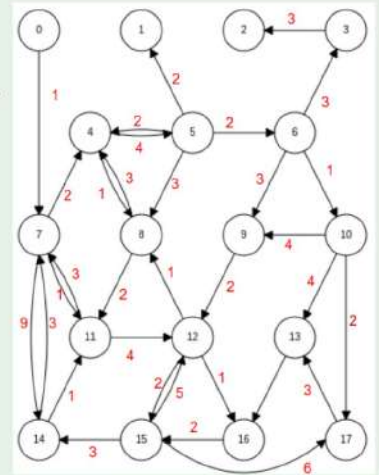
Exercises

- ❶ Run previous examples, with BFS **graph** search.
- ❷ Consider the following graph, initial state 0, goal state 17:
 - ❶ explore it using BFS, tree version
 - ❷ explore it using BFS, graph versionchildren should be expanded in numerical order
- ❸ Solve the 4-queen problem (incremental version) by BFS
 - children should be expanded in row-index order
 - do tree search and graph search differ?



Exercises

- 1 Apply UCS, graph version, to previous example
- 2 Apply UCS to the Romania-map Example (both tree and graph versions)
- 3 Consider the following graph, initial state 0, goal state 17
 - 1 explore it using UCS (tree version)
 - 2 explore it using UCS (graph version)



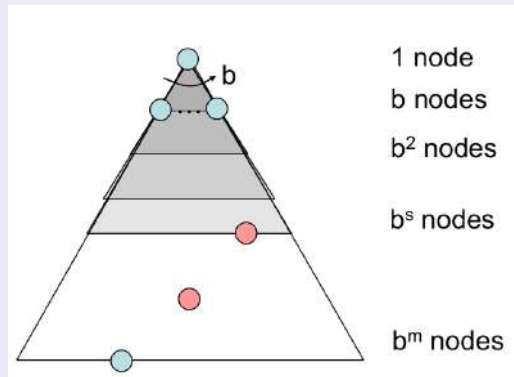
Exercises

- ❶ Run previous examples, with BFS **graph** search.
- ❷ Consider the following graph, initial state 0, goal state 17:
 - ❶ explore it using BFS, tree version
 - ❷ explore it using BFS, graph versionchildren should be expanded in numerical order
- ❸ Solve the 4-queen problem (incremental version) by BFS
 - children should be expanded in row-index order
 - do tree search and graph search differ?

Breadth-First Search (BFS): Properties

d : depth of shallowest solution

- How many steps?
 - processes all nodes above shallowest solution
 $\Rightarrow$ takes $O(b^d)$ time
- How much memory? (tree search)
 - max frontier size: b^d nodes
 $\Rightarrow$ $O(b^d)$ memory size
- Is it complete?
 - if solution exists, b^d finite
 $\Rightarrow$ Yes
- Is it optimal?
 - if and only if all costs are 1
 $\Rightarrow$ shallowest solution



(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement is a major problem for breadth-first search

Breadth-First Search (BFS): Time and Memory

- Assume:
 - 1 million nodes generated per second
 - 1 node requires 1000 bytes of storage
 - branching factor $b = 10$

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

(© S. Russell & P. Norwig, AIMA)

Memory requirements is a bigger problem for BFS than execution time

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies**
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search**
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Depth-First Search Strategy (DFS)

Depth-First Search

- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
⇒ novel successors pushed to the top of the stack

Depth-First Search Strategy (DFS)

Depth-First Search

- Idea: **Expand first the deepest unexpanded nodes**
- Implementation: **frontier/fringe** implemented as a LIFO queue (aka stack)
⇒ novel successors pushed to the top of the stack

Depth-First Search Strategy (DFS)

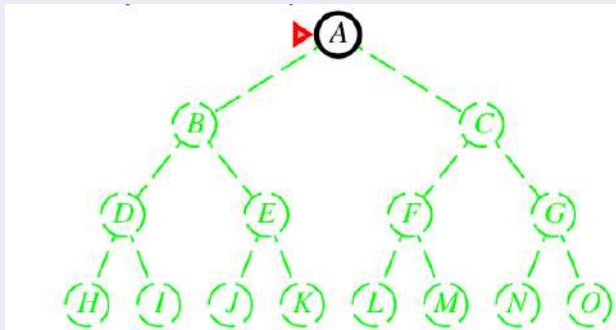
Depth-First Search

- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack

Depth-First Search Strategy (DFS)

Depth-First Search

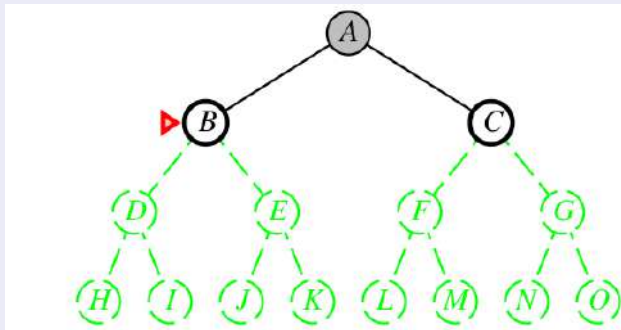
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

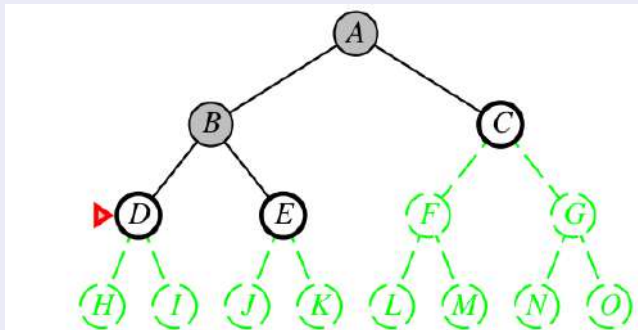
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

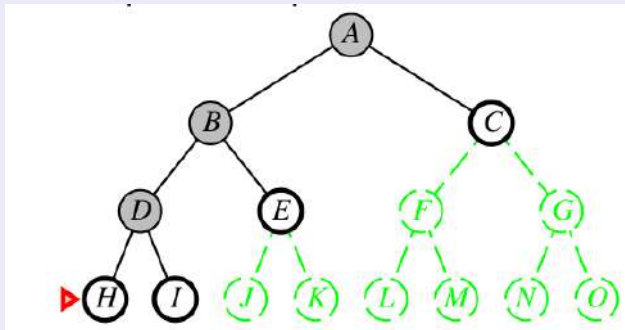
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

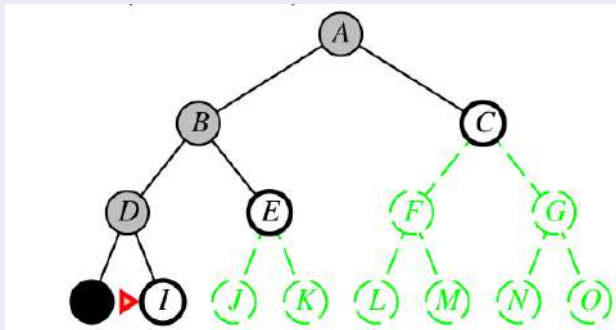
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

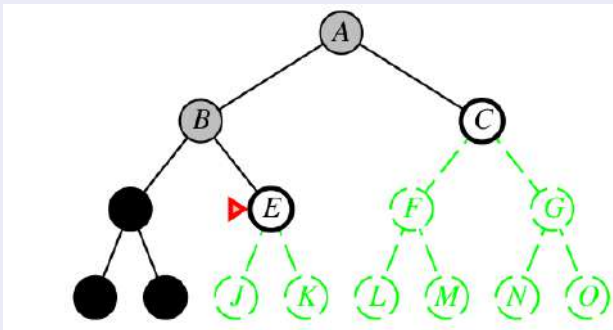
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

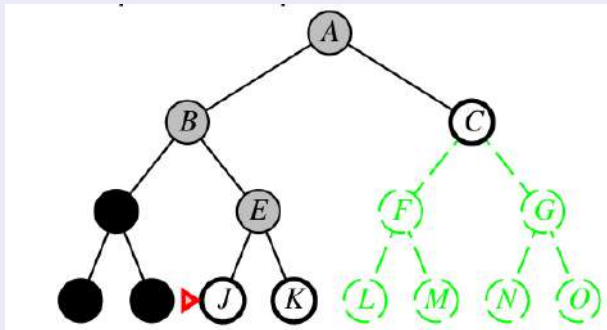
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

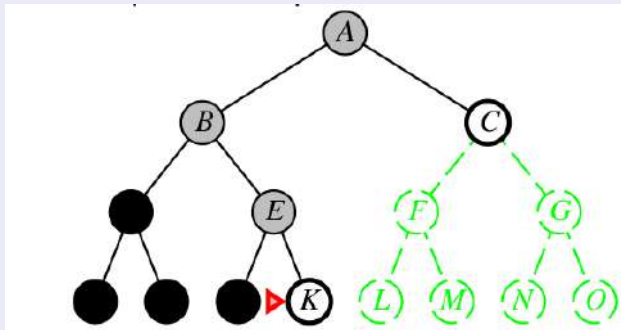
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

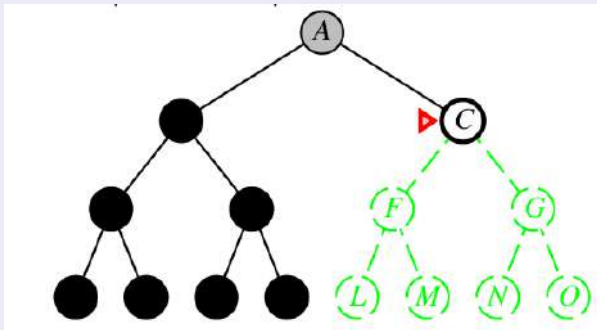
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

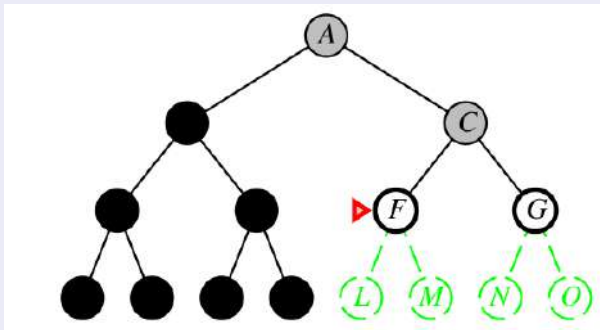
- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
 - ⇒ novel successors pushed to the top of the stack



Depth-First Search Strategy (DFS)

Depth-First Search

- Idea: Expand first the deepest unexpanded nodes
- Implementation: frontier/fringe implemented as a LIFO queue (aka stack)
⇒ novel successors pushed to the top of the stack



Depth-First Search

DFS, Graph version (Tree version without “reached”)

Similar to BFS, using a LIFO access for frontier/fringe rather than FIFO.

```
function DEPTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL) // early test
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a LIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child // early test
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

(© S. Russell & P. Norwig, AIMA)

Note: the goal test is applied to each node when it is generated, before being added to the frontier (early test), rather than when it is selected from the frontier for expansion

⇒ solution detected 1 layer earlier

Remarks

Order of action Expansion (EXPAND())

Due to the LIFO extraction strategy, the node extracted is the the **last** node yielded by EXPAND()
⇒ the nodes are explored in reverse order wrt. action expansion

DFS as Tree search

DFS typically implemented as **Tree** search
(otherwise we loose the benefit of polynomial memory usage, see later)

DFS vs UCS

In principle, DFS could be implemented as UCS ("best first") with $cost = -1$ for each action, but

- Access to LIFO queue ($O(1)$) faster than that to priority queue ($O(\log(n))$)
- UCS typically graph search, DFS typically tree search
- Beware of:
 - UCS performs "late test": when extracting node from the frontier
 - DFS performs "early test": when inserting node into the frontier

⇒ Implementing DFS as UCS would be much less efficient

EJEMPLO

Si exploras en orden alfabético, exploras
en reversed alphabetical order

Remarks

Order of action Expansion (EXPAND())

Due to the LIFO extraction strategy, the node extracted is the the **last** node yielded by EXPAND()
⇒ the nodes are explored in reverse order wrt. action expansion

DFS as Tree search

DFS typically implemented as **Tree** search
(otherwise we loose the benefit of polynomial memory usage, see later)

DFS vs UCS

In principle, DFS could be implemented as UCS (“best first”) with $cost = -1$ for each action, but

- Access to LIFO queue ($O(1)$) faster than that to priority queue ($O(\log(n))$)
- UCS typically graph search, DFS typically tree search
- **Beware of goal test!**
 - UCS performs “late test”: when extracting node from the frontier
 - DFS performs “early test”: when inserting node into the frontier

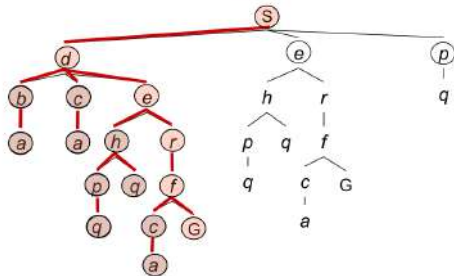
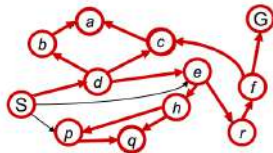
⇒ Implementing DFS as UCS would be much less efficient

Depth-First Search

DFS, (tree version, children expanded in reverse alphabetical order)

Strategy: expand a deepest node first

Implementation:
Fringe is a LIFO stack



node	stack
	{S}
S	{d, e, p}
d	{b, c, e, e, p}
b	{a, c, e, e, p}
a	{c, e, e, p}
c	{a, e, e, p}
a	{e, e, p}
e	{h, r, e, p}
h	{p, q, r, e, p}
p	{q, q, r, e, p}
q	{q, r, e, p}
q	{r, e, p}
r	{f, e, p}
f	{c, G, e, p} Goal reached!

DFS Example

DFS Search for 8-Puzzle (Tree search)

Expansion order: (move empty cell) down, right, left, up.



Frontier



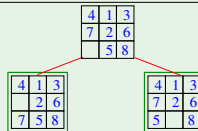
Goal

4	1	3
7	2	6
	5	8

DFS Example

DFS Search for 8-Puzzle (Tree search)

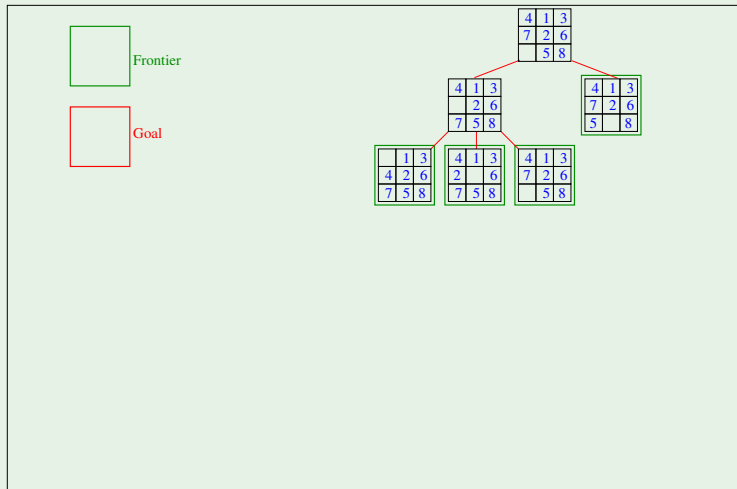
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

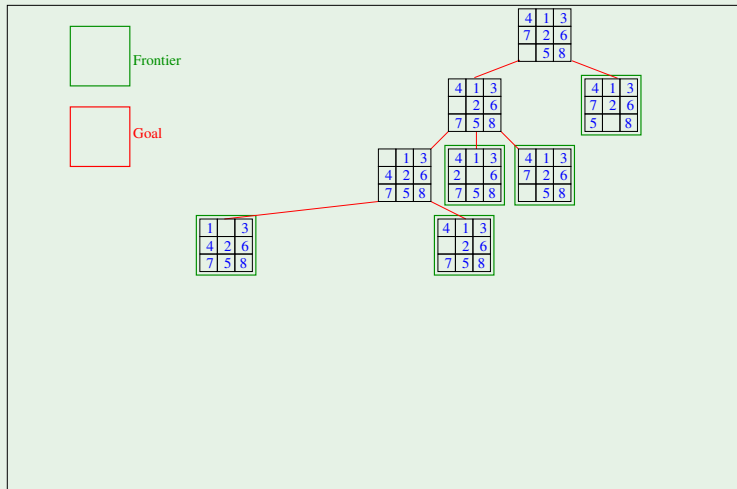
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

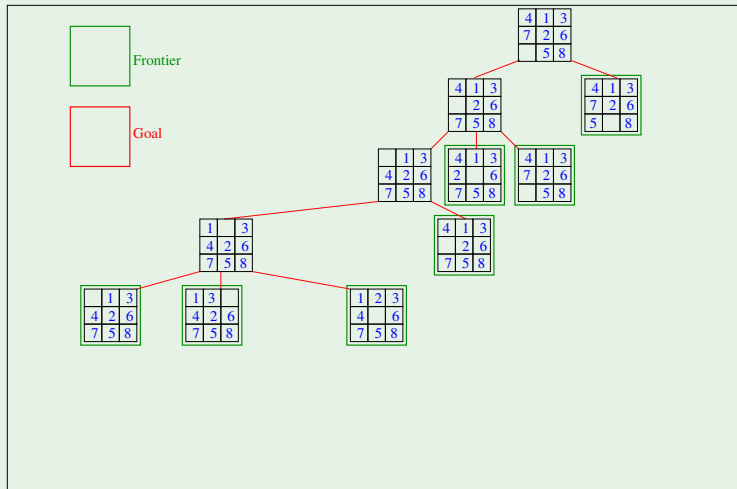
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

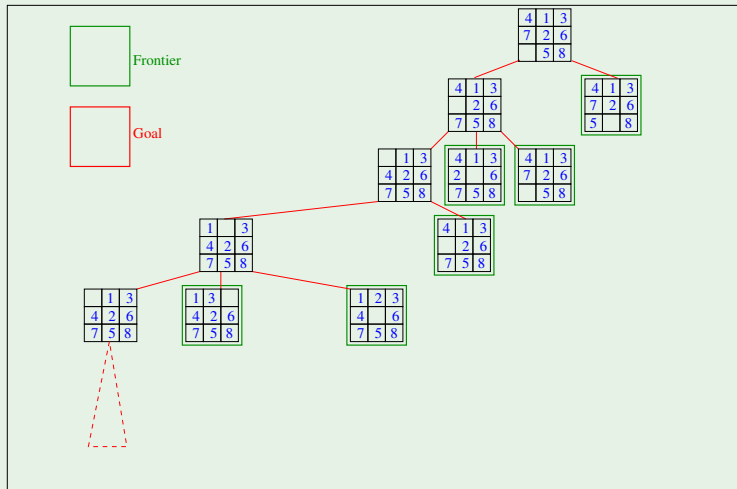
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

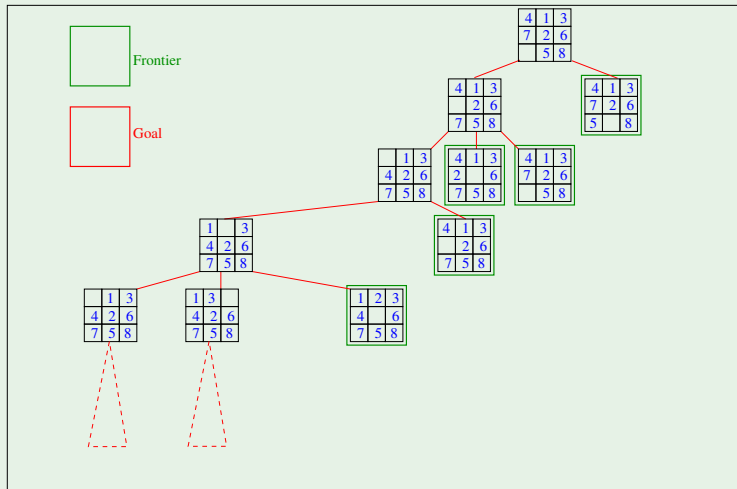
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

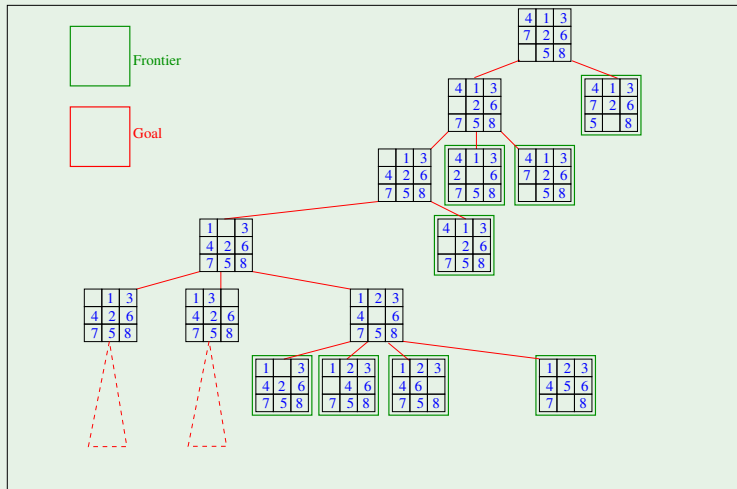
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

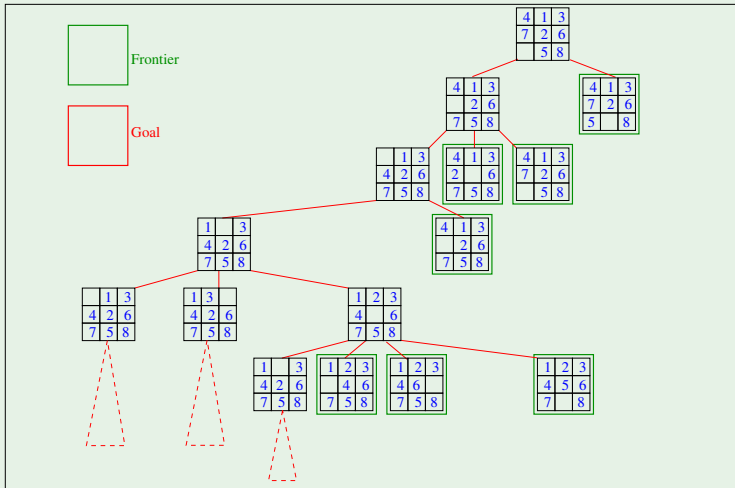
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

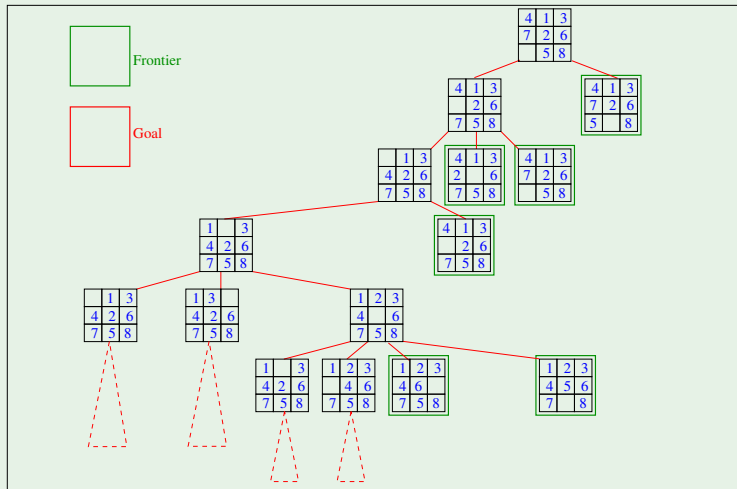
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

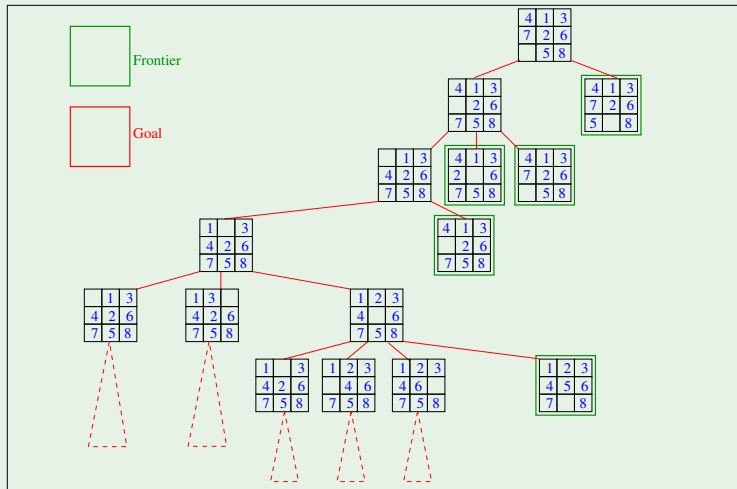
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

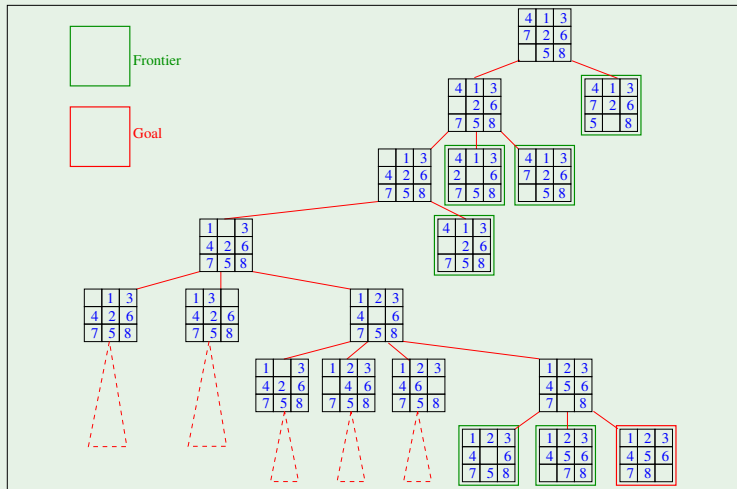
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

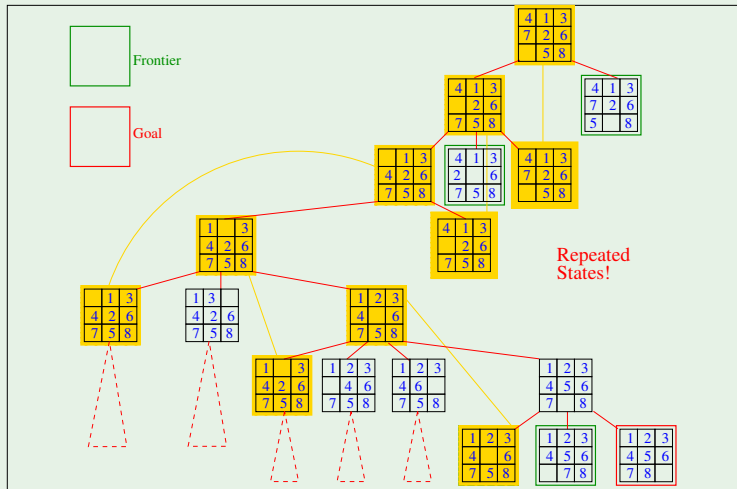
Expansion order: (move empty cell) down, right, left, up.



DFS Example

DFS Search for 8-Puzzle (Tree search)

Expansion order: (move empty cell) down, right, left, up.



Exercises

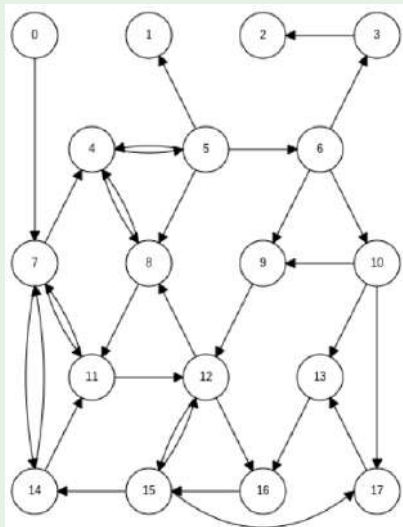
- 1 As with previous examples, with loop detection.
- 2 As with previous examples, with DFS **graph** search.
- 3 Consider the following graph, initial state 0, goal state 17:
 - explore it using DFS, tree version
 - explore it using DFS, graph versionchildren should be expanded in numerical order
- 4 Solve the 4-queen problem (incremental version) by DFS
 - children should be expanded in row-index order
 - do tree search and graph search differ?

Exercises

- 1 As with previous examples, with loop detection.
- 2 As with previous examples, with DFS **graph** search.
- 3 Consider the following graph, initial state 0, goal state 17:
 - explore it using DFS, tree version
 - explore it using DFS, graph versionchildren should be expanded in numerical order
- 4 Solve the 4-queen problem (incremental version) by DFS
 - children should be expanded in row-index order
 - do tree search and graph search differ?

Exercises

- ❶ As with previous examples, with loop detection.
- ❷ As with previous examples, with DFS **graph** search.
- ❸ Consider the following graph, initial state 0, goal state 17:
 - ❶ explore it using DFS, tree version
 - ❷ explore it using DFS, graph versionchildren should be expanded in numerical order
- ❹ Solve the 4-queen problem (incremental version) by DFS
 - children should be expanded in row-index order
 - do tree search and graph search differ?

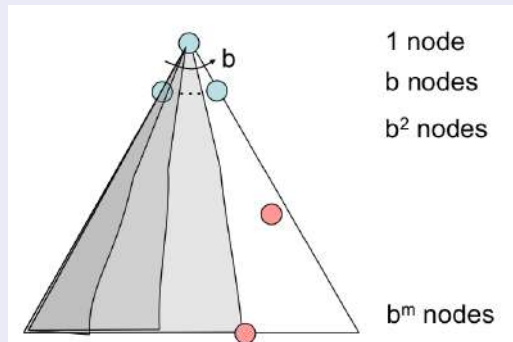


Exercises

- ① As with previous examples, with loop detection.
- ② As with previous examples, with DFS **graph** search.
- ③ Consider the following graph, initial state 0, goal state 17:
 - ① explore it using DFS, tree version
 - ② explore it using DFS, graph versionchildren should be expanded in numerical order
- ④ Solve the 4-queen problem (incremental version) by DFS
 - children should be expanded in row-index order
 - do tree search and graph search differ?

Depth-First Search (DFS): Properties

- How many steps?
 - could process the whole tree!
⇒ if m finite, takes $O(b^m)$ time
- How much memory?
 - only siblings on path to root
⇒ $O(bm)$ memory size
- Is it complete?
 - if infinite state space: **no**
 - if finite state space:
 - graph version: **yes**
 - tree version: **only if we prevent loops**
(cheap loop test)
- Is it optimal?
 - No, regardless of depth/cost
⇒ “leftmost” solution



(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement much better than BFS: $O(bm)$ vs. $O(b^d)$!
⇒ typically preferred to BFS

A Variant of DFS: Backtracking Search

Backtracking Search

- Idea: **only one successor is generated at the time**
 - each partially-expanded node remembers which successor to generate next
 - generate a successor by modifying the current state description, rather than copying it first
 - Applied in CSP, SAT/SMT and Logic Programming

⇒ only $O(m)$ memory is needed rather than $O(bm)$

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies**
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening**
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Depth-Limited Search (DLS) Strategy

Depth-Limited Search (DLS)

- Idea: **depth-first search with depth limit l**
 - i.e., nodes at depth l treated as having no successors
 - DFS is DLS with $l = +\infty$
- **solves the infinite-path problem of DFS**
 $\implies$ allows DFS deal with infinite-state spaces
- **useful also if maximum-depth is known by domain knowledge**
 - e.g., if maximum node distance in a graph (**diameter**) is known
 - Ex: **Romania trip**: 9 steps
- Drawbacks (d : depth of the shallowest goal):
 - if $d > l \implies$ **incomplete**
 - if $d < l \implies$ **takes $O(b^l)$ instead of $O(b^d)$ steps**

Depth-Limited Search (DLS) Strategy [cont.]

DLS (tree version)

```
function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure or cutoff  
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element  
  result  $\leftarrow$  failure  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    if DEPTH(node) >  $\ell$  then  
      result  $\leftarrow$  cutoff  
    else if not IS-CYCLE(node) do  
      for each child in EXPAND(problem, node) do  
        add child to frontier  
  return result
```

(© S. Russell & P. Norwig, AIMA)

Note: the distinction between early goal test and late goal test is less relevant here

Iterative-Deepening Search Strategy (IDS)

Iterative-Deepening Search

- Idea: call iteratively DLS for increasing depths $l = 0, 1, 2, 3, \dots$
- combines the advantages of breadth- and depth-first strategies
 - complete (like BFS)
 - takes $O(b^d)$ steps (like BFS)
 - requires $O(bd)$ memory (like DFS)
 - explores a single branch at a time (like DFS)
 - optimal only if step cost = 1
 - optimal variants exist: [iterative-lengthening search](#) (see AIMA)
- The favorite search strategy when the search space is very large and depth is not known

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result
```

Iterative-Deepening Search (IDS) [cont.]

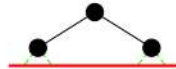
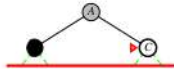
Limit = 0



(© S. Russell & P. Norwig, AIMA)

Iterative-Deepening Search (IDS) [cont.]

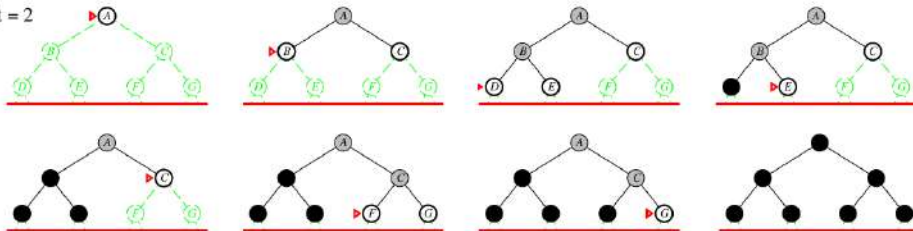
Limit = 1



(© S. Russell & P. Norwig, AIMA)

Iterative-Deepening Search (IDS) [cont.]

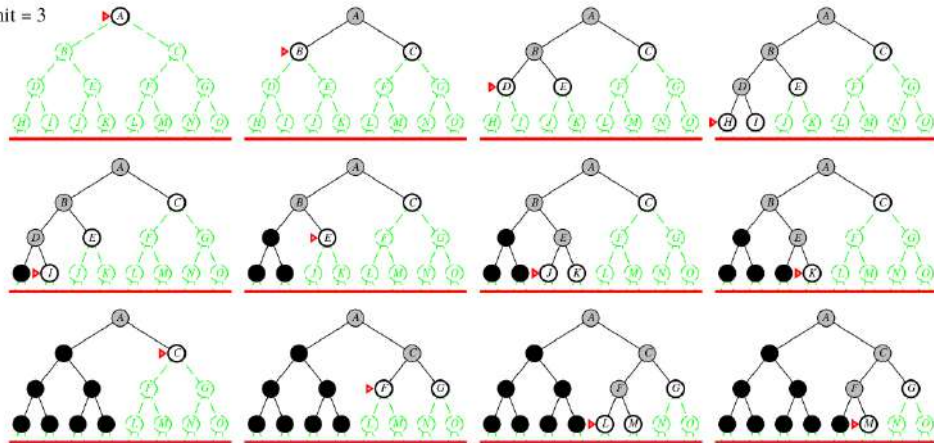
Limit = 2



(© S. Russell & P. Norwig, AIMA)

Iterative-Deepening Search (IDS) [cont.]

Limit = 3



(© S. Russell & P. Norvig, AIMA)

Iterative-Deepening Search Strategy (IDS) [cont.]

Remark: Why “only” $O(b^d)$ steps?

- may seem wasteful since states are generated multiple times...
- ... however, only a small fraction of nodes are multiply generated
- number of repeatedly-generated nodes decreases exponentially with number of repetitions
 - depth 1 (b nodes): repeated d times
 - depth 2 (b^2 nodes): repeated $d - 1$ times
 - ...
 - depth d (b^d nodes): repeated 1 time

⇒ The total number of generated nodes is:

$$N(IDS) = (d)b^1 + (d-1)b^2 + \dots + (1)b^d = O(b^d)$$

$$N(BFS) = b^1 + b^2 + \dots + b^d = O(b^d)$$

- Ex: with $b = 10$ and $d = 5$:

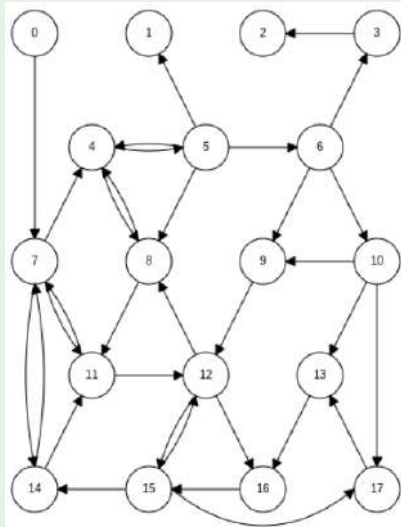
$$N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,000$$

$$N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110$$

⇒ not significantly worse than BFS

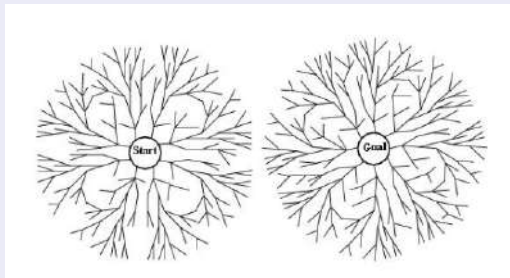
Exercises

- ➊ Consider the following graph, initial state 0, goal state 17:
 - ➊ explore it using IDS, tree version
 - ➋ explore it using IDS, graph versionchildren should be expanded in numerical order
- ➋ Solve the 4-queen problem (incremental version) by ITS
 - children should be expanded in row-index order
- ➌ Try to solve the 8-puzzle problem by ITS (max 3-4 layers)



Bidirectional Search [hints]

- Idea: Two simultaneous searches:
 - forward: from start node
 - backward: from goal nodechecking if the node belongs to the other frontier before expansion
- Rationale: $b^{d/2} + b^{d/2} \ll b^d$
 $\Rightarrow$ number of steps and memory consumption are $\approx 2b^{d/2}$
- backward search can be tricky in some cases (e.g. 8-queens)



Uninformed Search Strategies: Comparison

Evaluation of tree-search strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

^a: complete if b is finite

^b: complete if step costs $\geq \epsilon$ for some positive ϵ

^c: optimal if step costs are all identical

^d: if both directions use breadth-first search

(© S. Russell & P. Norwig, AIMA)

For graph searches, the main differences are:

- depth-first search is complete for finite-state spaces
- space & time complexities are bounded by the state space size

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies**
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Informed Search Strategies

Some general principles

- The intelligence of a system cannot be measured only in terms of search capacity, but in **the ability to use knowledge about the problem to reduce/mitigate the combinatorial explosion**
- If the system has some control on the order in which candidate solutions are generated, then it is useful to use this order so that actual solutions have a high chance to appear earlier
- For a system with limited processing capacity:
intelligence is the wise choice of what to do next

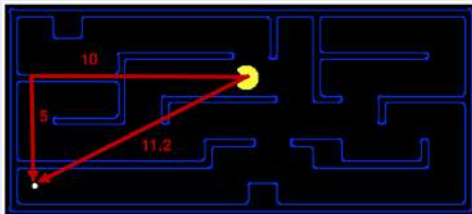
Heuristic search and heuristic functions

Heuristic search and heuristic functions

- Uninformed UCS strategy ignores the goal when selecting nodes

⇒ Idea: don't ignore the goal when selecting nodes

- Intuition: often extra knowledge can be used to guide the search towards the goal: heuristics
- A heuristic is a function $h(n)$ that estimates how close a state n is to a goal
 - designed for a particular search problem
 - Ex Manhattan distance, Euclidean distance for pathing



nombre
manhattan!

(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

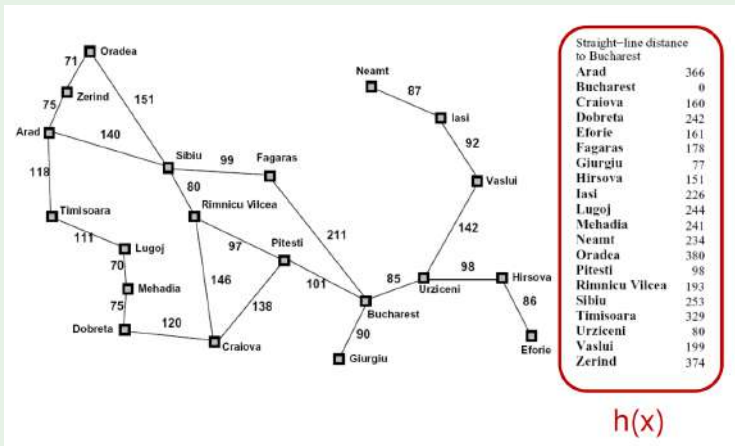
Best-first Search Strategies

General approach of informed search: Best-first search

- **Best-first search**: node selected for expansion based on an **evaluation function $f(n)$**
 - represent a **cost estimate** $\implies$ choose node which appears best
 - implemented like uniform-cost search, with f instead of g
 - $\implies$ the frontier is a priority queue sorted in decreasing order of $f(n)$
 - both tree-based and graph-based versions
 - most often f includes a **heuristic function $h(n)$**
- **Heuristic function $h(n) \in \mathbb{R}^+$** :
estimated cost of the cheapest path from the state at node n to a goal state
 - $h(n) \geq 0 \forall n$
 - If G is goal, then $h(G) = 0$
 - implements **extra domain knowledge**
 - **depends only on state, not on node** (e.g., independent on paths)
- **Main strategies**:
 - **Greedy best-first search**
 - **A^* search**

Example: Straight-Line Distance $h_{SLD}(n)$

- $h(n) \stackrel{\text{def}}{=} h_{SLD}(n)$: straight-line distance heuristic
 - different from actual minimum-path distance
 - cannot be computed from the problem description itself



Outline

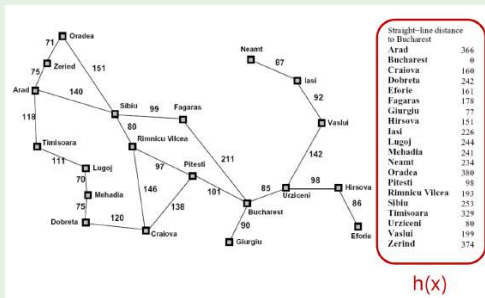
- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies **MON SEP 21**
 - Greedy Best-First Search
 - A* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

Greedy Best-First Search Strategy (GBFS)

Greedy Best-First Search (aka Greedy Search)

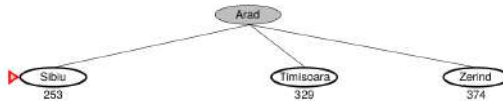
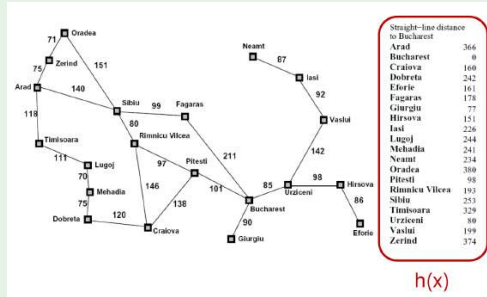
- Idea: Expand first the node n with lowest estimate cost to the closest goal, $h(n)$
- Implementation: same as uniform-cost search, with $g(n) \stackrel{\text{def}}{=} h(n)$
 - ⇒ expands the node that appears to be closest to goal
- both tree and graph versions

Greedy Best-First Search Strategy: Example

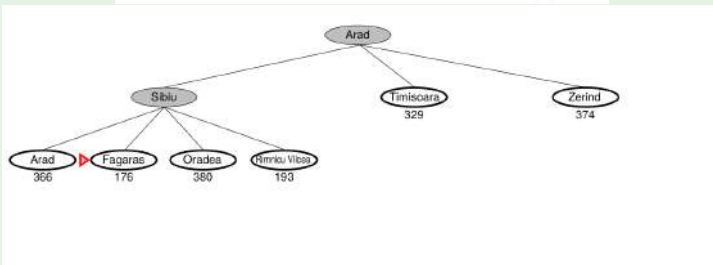
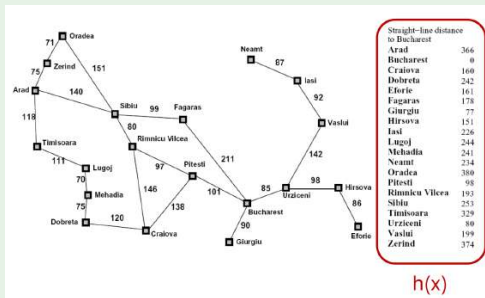


Arad
366

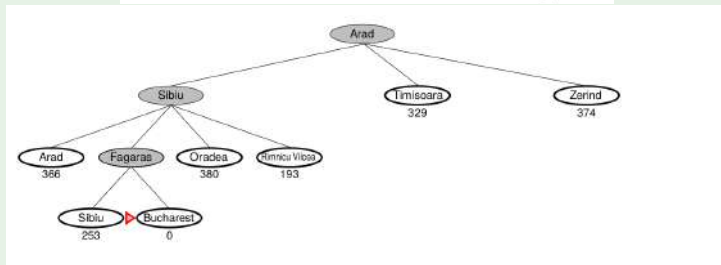
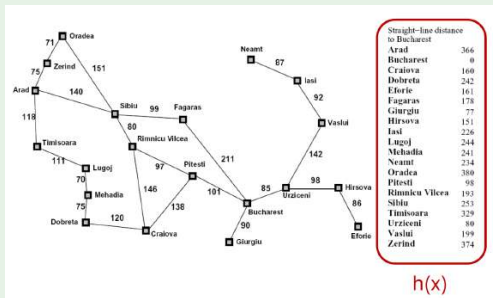
Greedy Best-First Search Strategy: Example



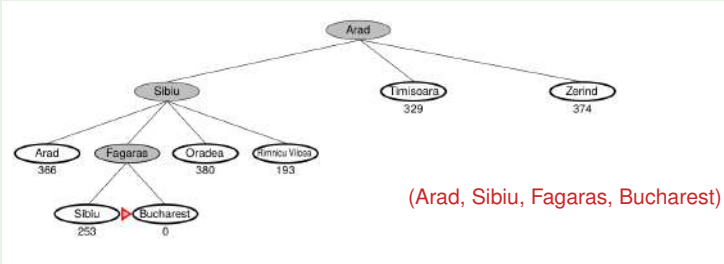
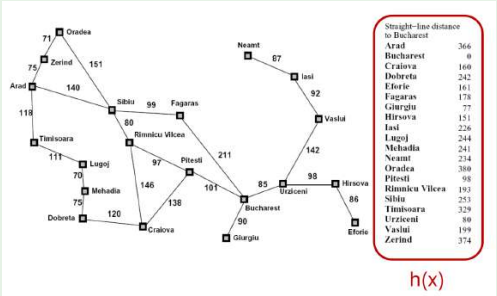
Greedy Best-First Search Strategy: Example



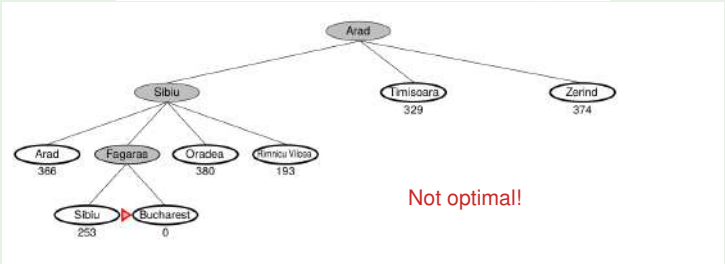
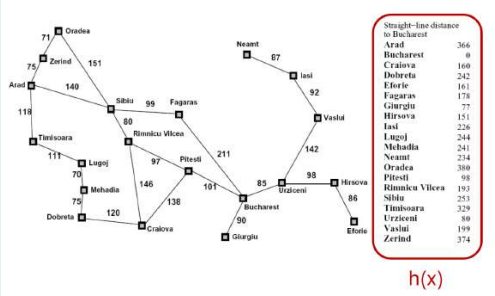
Greedy Best-First Search Strategy: Example



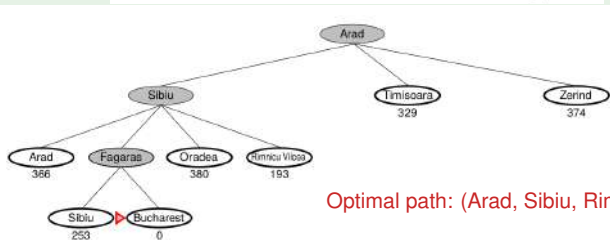
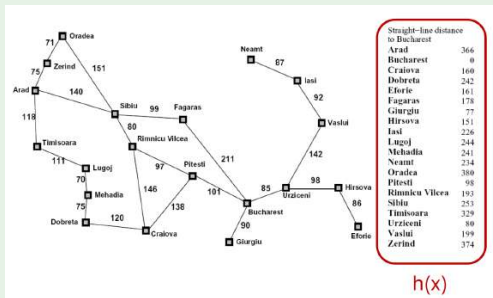
Greedy Best-First Search Strategy: Example



Greedy Best-First Search Strategy: Example

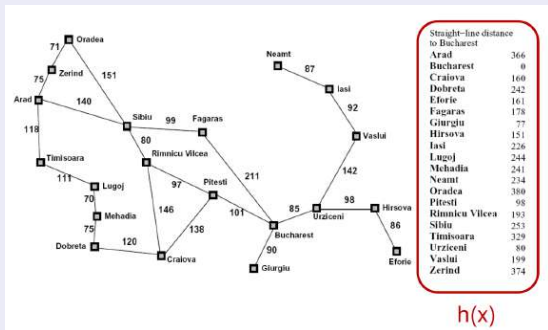


Greedy Best-First Search Strategy: Example



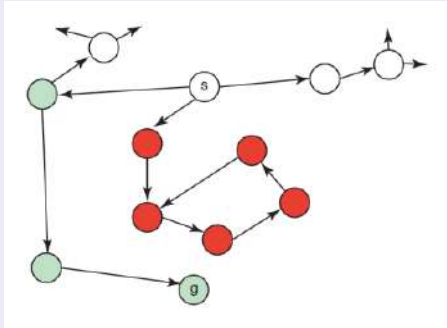
Greedy Best-First Search: (Non-)Optimality

- Greedy best-first search is not optimal
 - it is not guaranteed to find the best solution
 - it is not guaranteed to find the best path toward a solution
- picks the node with minimum (estimated) distance to goal, regardless the cost to reach it
 - Ex: when in Sibiu, it picks Fagaras rather than Rimnicu Vitea



Greedy Best-First Search: (In-)Completeness

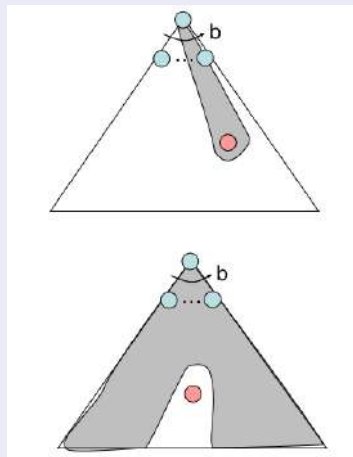
- Tree-based Greedy best-first search is not complete
 - may lead to infinite loops
- Graph-based version complete (if state space finite)
- substantially same completeness issues as DFS



(Courtesy of Maria Simi, UniPI)

Greedy Best-First Search (GBFS): Properties

- How many steps?
 - in worst cases may explore all states
⇒ takes $O(b^d)$ time
 - if good heuristics:
⇒ may give good improvements
- How much memory?
 - max frontier size: b^d (as with UCS)
⇒ $O(b^d)$ memory size
- Is it complete?
 - tree: no
 - graph: yes if space finite
- Is it optimal?
 - No

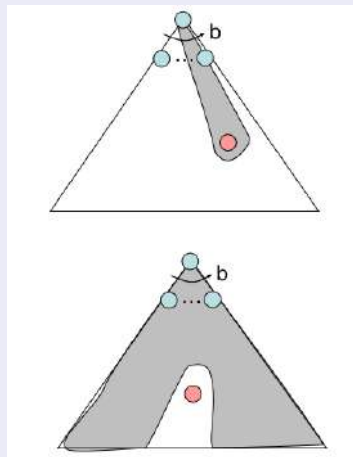


(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement is a major problem also for GBFS

Greedy Best-First Search (GBFS): Properties

- How many steps?
 - in worst cases may explore all states
⇒ takes $O(b^d)$ time
 - if good heuristics:
⇒ may give good improvements
- How much memory?
 - max frontier size: b^d (as with UCS)
⇒ $O(b^d)$ memory size
- Is it complete?
 - tree: no
 - graph: yes if space finite
- Is it optimal?
 - No

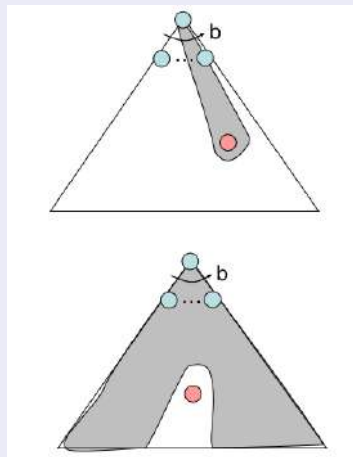


(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement is a major problem also for GBFS

Greedy Best-First Search (GBFS): Properties

- How many steps?
 - in worst cases may explore all states
⇒ takes $O(b^d)$ time
 - if good heuristics:
⇒ may give good improvements
- How much memory?
 - max frontier size: b^d (as with UCS)
⇒ $O(b^d)$ memory size
- Is it complete?
 - tree: no
 - graph: yes if space finite
- Is it optimal?
 - No



(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement is a major problem also for GBFS

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies**
 - Greedy Best-First Search
 - A* Search**
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions

A* Search Strategy

A* Search

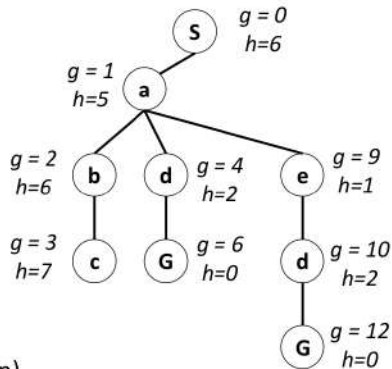
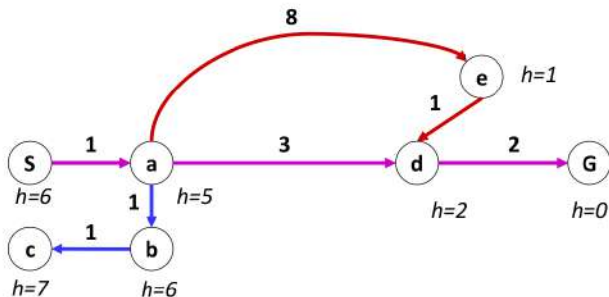
- Best-known form of best-first search
- Idea: avoid expanding paths that are already expensive
- Combine Uniform-Cost and Greedy search: $f(n) = g(n) + h(n)$
 - $g(n)$: cost so far to reach n
 - $h(n)$: estimated cost to goal from n
 - $f(n)$: estimated total cost of path through n to goal

⇒ Expand first the node n with lowest estimated cost of the cheapest solution through n

- Implementation: same as uniform-cost search, with $g(n) + h(n)$ instead of $g(n)$

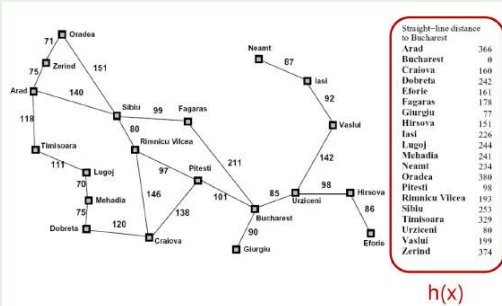
A* Search Strategy: Example

- Uniform-cost orders by path cost, or *backward cost* $g(n)$
- Greedy orders by goal proximity, or *forward cost* $h(n)$



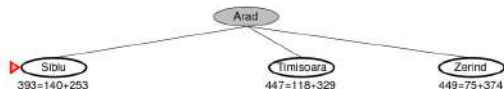
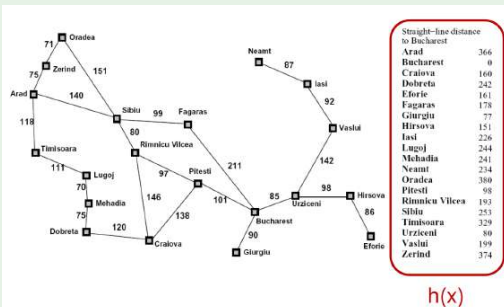
- A* Search orders by the sum: $f(n) = g(n) + h(n)$

A* Search Strategy: Example

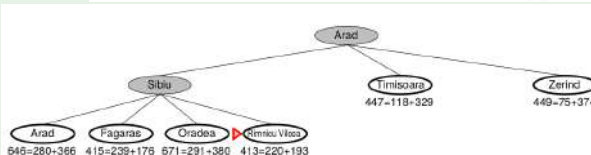
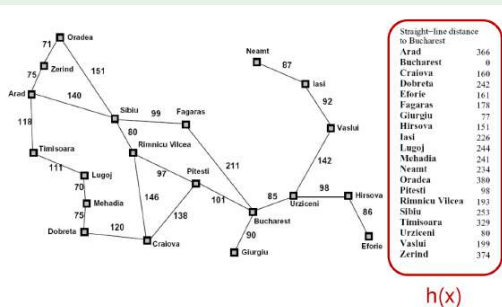


▶ Arad
366-0=366

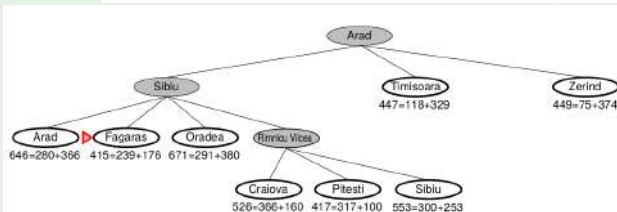
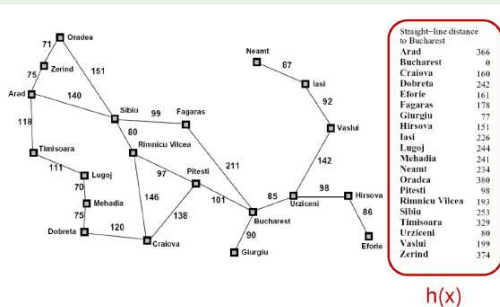
A* Search Strategy: Example



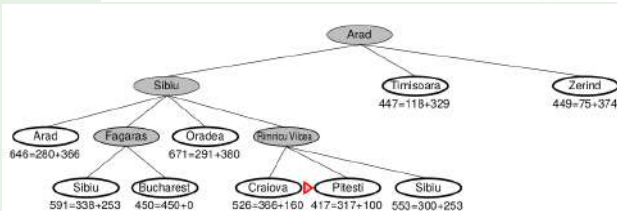
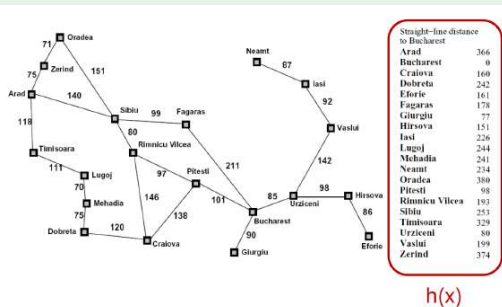
A* Search Strategy: Example



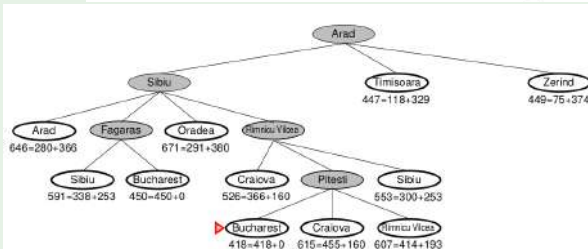
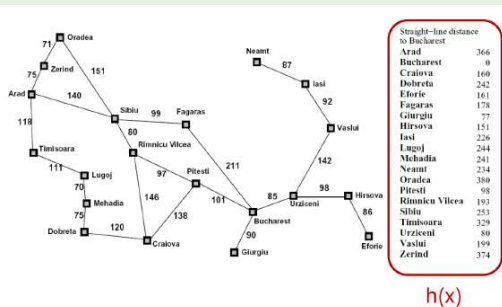
A* Search Strategy: Example



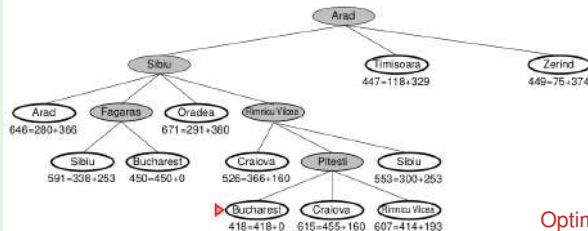
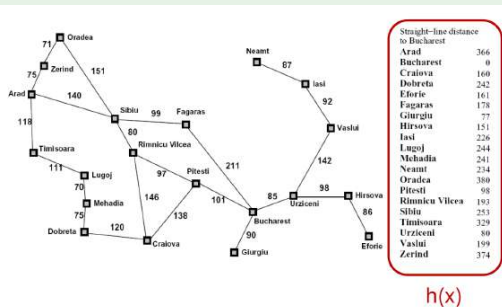
A* Search Strategy: Example



A* Search Strategy: Example



A* Search Strategy: Example



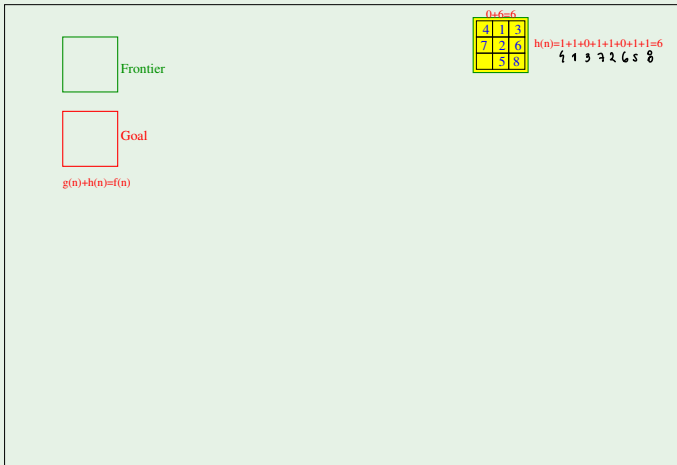
Optimal path!

Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.

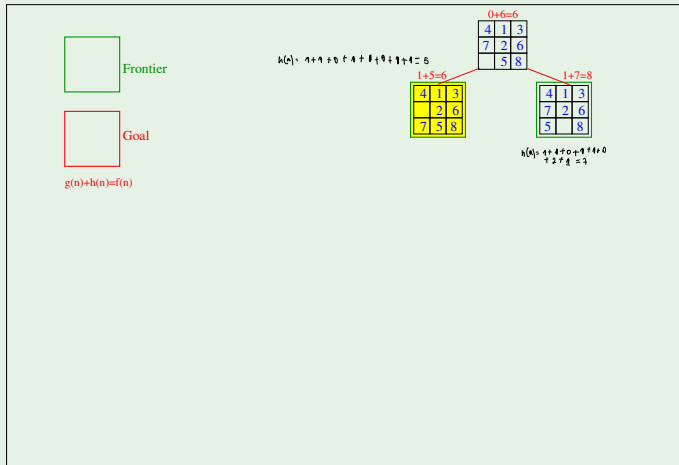


Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.

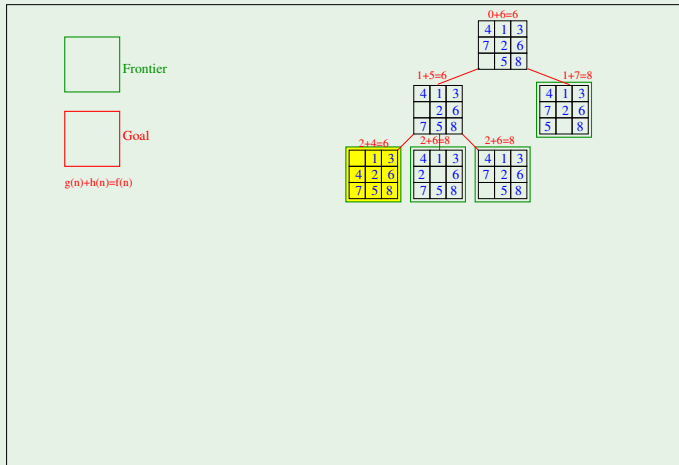


Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.

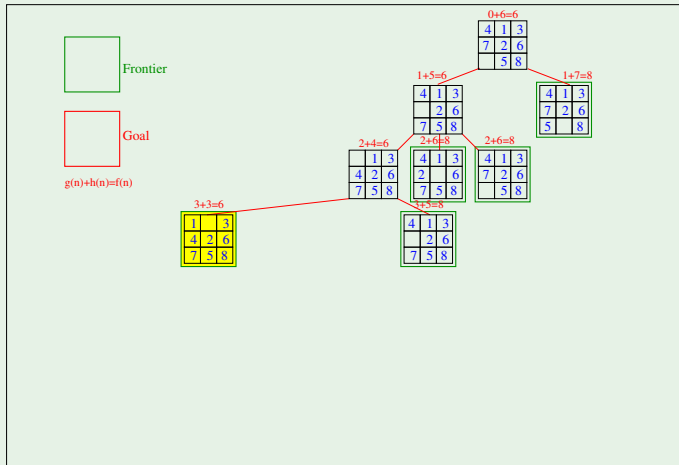


Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.

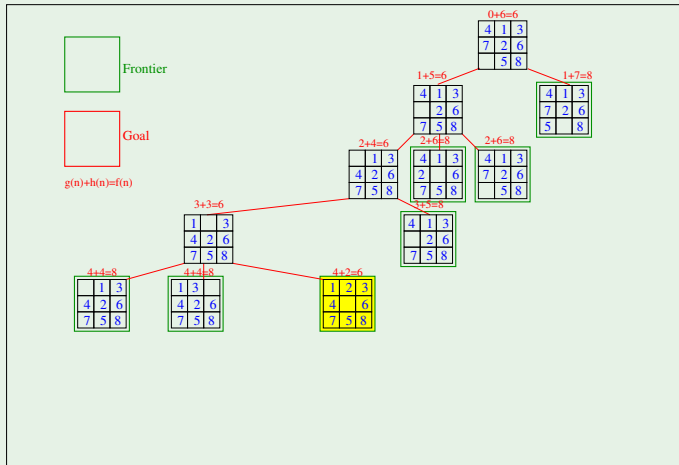


Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.

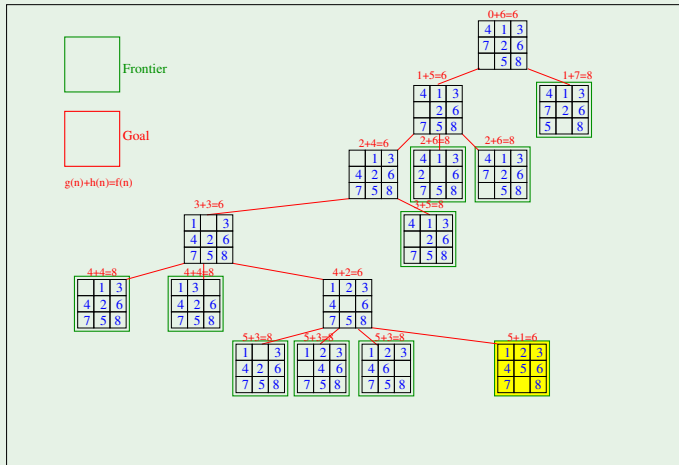


Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.

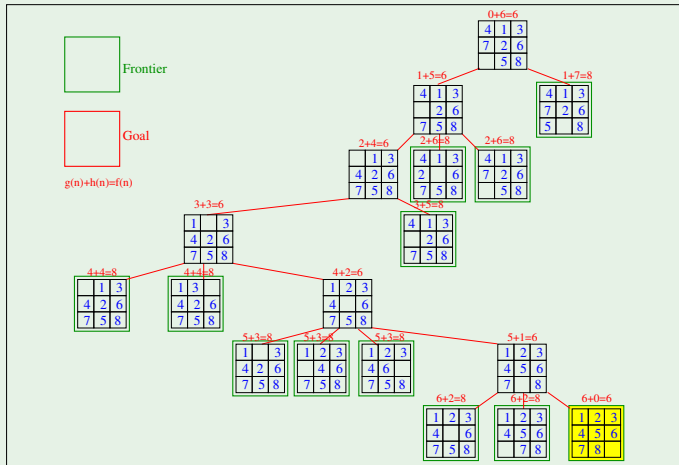


Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.

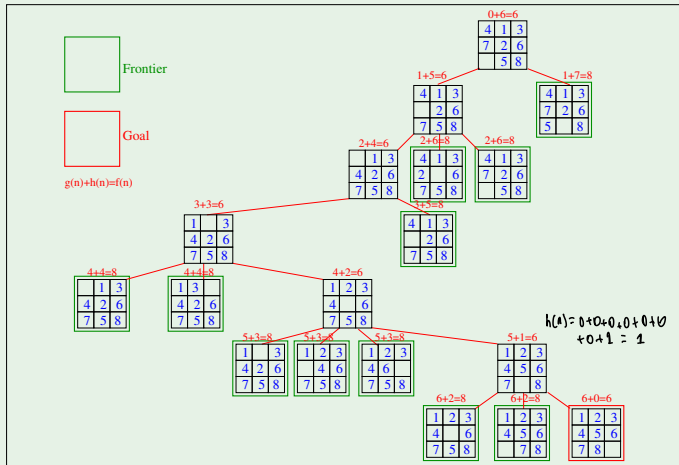


Example

A* Search for 8-Puzzle

$g(n)$ =depth; $h(n)$ = sum of manhattan distances of tiles to their goal position (see later)

Expansion order: (move empty cell) up, left, right, down.



Exercise

- Modify the Romanian Map example as follows:
 - drop the arcs from Faragas to Bucharest and from Pitesti to Bucharest
 - add one arc from Oradea to Neamt of length 250.
- Execute the A^* algorithm from Arad to Bucharest with such new map

A* Search: Admissible and Consistent Heuristics

Admissible heuristics $h(n)$

- $h(n)$ is **admissible** (aka **optimistic**) iff it never overestimates the cost to reach the goal:
 - $h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost from n
 - ex: the straight-line distance $h_{SDL}()$ to Bucharest

Consistent heuristics $h(n)$

- $h(n)$ is **consistent** (aka **monotonic**) iff, for every successor n' of n generated by any action a with step cost $c(n, a, n')$,
 $h(n) \leq c(n, a, n') + h(n')$
 - verifies the triangular inequality
 - ex: the straight-line distance $h_{SDL}()$ to Bucharest



Property

If $h(n)$ is consistent, then $h(n)$ is admissible.

A* Search: Properties and Optimality

Optimality of A* tree search

best path $h(n) + g(n)$

If $h(n)$ is admissible, then A* tree search is optimal

Properties of A* graph search

- If $h(n)$ is consistent, then $f(n)$ is non-decreasing along any path
- Graph A* selects node n from the frontier only if the optimal path to n has been found

⇒ A* (graph search) expands nodes in non-decreasing order of f

Optimality of A* graph search

If $h(n)$ is consistent, then A* graph search is optimal

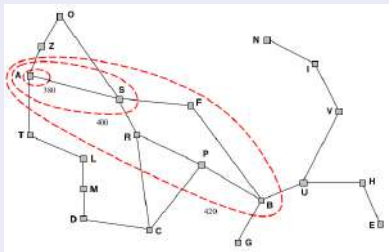
Note

As soon as we progress along an optimal path, h (and hence f) becomes progressively less optimistic and more realistic.

A* Graph Search Optimality: Intuitions

If $h(n)$ is consistent, then A* graph search is optimal

- A* expands nodes in order of non-decreasing f value
- Gradually adds “ f -contours” of nodes (as BFS adds layers)
 - contour i has all nodes with $f = f_i$, s.t. $f_i < f_{i+1}$
 - cannot expand contour f_{i+1} until contour f_i is fully expanded
- If C^* is the cost of the optimal solution path
 - 1 A* expands all nodes s.t. $f(n) < C^*$
 - 2 A* might expand some of the nodes on “goal contour” s.t. $f(n) = C^*$ before selecting a goal node.
 - 3 A* does not expand nodes s.t. $f(n) > C^*$ (pruning)

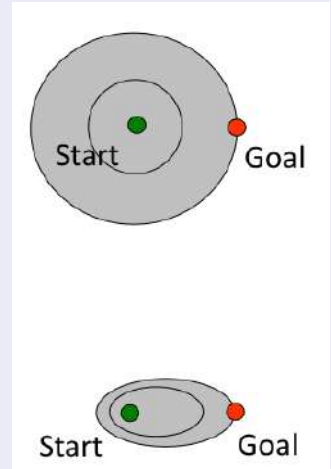


UCS vs A* Contours

Intuition

- UCS expands equally in all “directions”
- A* expands mainly toward the goal

*has a aca
lines*



A* Search: Completeness

If all step costs exceed some finite ϵ and b is finite,
then there are only finitely many nodes n s.t. $f(n) \leq C^*$
 $\implies$ A* is complete.

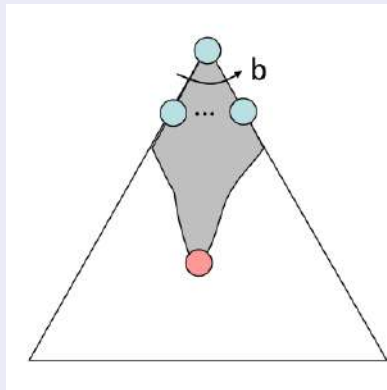
A* Search: Properties

(Under simplified hypotheses) it can be shown the following.

Let $\epsilon \stackrel{\text{def}}{=} (h^* - h)/h^*$ (relative error)

b^ϵ : effective branching factor

- How many steps?
 - takes $O((b^\epsilon)^d)$ time
 - if good heuristics, may give dramatic improvements
- How much memory?
 - Keeps all nodes in memory
 - $\Rightarrow O((b^\epsilon)^d)$ memory size (exponential, like UCS)
- Is it complete?
 - yes
- Is it optimal?
 - yes



(© D. Klein, P. Abbeel, S. Levine, S. Russell, U. Berkeley)

Memory requirement is a major problem also for A*

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies
 - Greedy Best-First Search
 - A* Search
 - **Memory-bounded Heuristic Search (hints)**
 - Heuristic Functions

Memory-bounded Heuristic Search (hints)

Some solutions to A^* space problems
(maintain completeness and optimality)

- Iterative-deepening A^* (IDA^*)
 - here cutoff information is the f-cost ($g+h$) instead of depth
- Recursive best-first search (RBFS)
 - attempts to mimic standard best-first search with linear space
- (simple) Memory-bounded A^* ((S)MA*)
 - drop the worst-leaf node when memory is full

Outline

- 1 Problem-Solving Agents
- 2 Some Example Problems
- 3 Search Generalities
 - Tree Search
 - Graph Search
 - Implementation Issues & Strategies
- 4 Uninformed Search Strategies
 - Best-first/Uniform-cost Search
 - Breadth-First Search
 - Depth-First Search
 - Depth-Limited Search & Iterative Deepening
- 5 Informed Search Strategies**
 - Greedy Best-First Search
 - A^* Search
 - Memory-bounded Heuristic Search (hints)
 - Heuristic Functions**

Admissible Heuristics

Main problem

What is the best admissible/consistent heuristic?

Dominance of Admissible Heuristics

Dominance

Let $h_1(n)$, $h_2(n)$ admissible heuristics.

- $h_2(n)$ dominates $h_1(n)$ iff $h_2(n) \geq h_1(n)$ for all n .

$\Rightarrow h_2(n)$ is better for search

- is nearer to $h^*(n)$

Let $h_1(n)$, $h_2(n)$ admissible heuristics. Let $h_{12} \stackrel{\text{def}}{=} \max(h_1(n), h_2(n))$.

- h_{12} is also admissible
- h_{12} dominates both $h_1(n)$, $h_2(n)$

Quality of Heuristics

Effective branching factor

- **Effective branching factor b^*** : the branching factor that a uniform tree of depth d would have in order to contain $N+1$ nodes

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

N being the number of nodes generated by the A^* search

- ex: if $d=5$ and $N = 52$, then $b^* = 1.92$
- experimental measure of b^* is fairly constant for hard problems
⇒ can provide a good guide to the heuristic's overall usefulness
- Ideal value of b^* is 1

Admissible Heuristics: Example [cont.]

Average performances on 100 random samples of 8-puzzle

Iterative-deepening search (IDS) vs. A^*

	Search Cost (nodes generated)			Effective Branching Factor		
d	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	–	539	113	–	1.44	1.23
16	–	1301	211	–	1.45	1.25
18	–	3056	363	–	1.46	1.26
20	–	7276	676	–	1.47	1.27
22	–	18094	1219	–	1.48	1.28
24	–	39135	1641	–	1.48	1.26

(© S. Russell & P. Norwig, AIMA)

⇒ Dramatic performance improvement

Admissible Heuristics from Relaxed Problems

Idea: Admissible heuristics can be derived from the exact solution cost of a relaxed version of the problem

- **Relaxed 8-puzzle:** a tile can move from any tile to any other tile
⇒ $h_1(n)$ gives the shortest solution
- **Relaxed 8-puzzle:** a tile can move to any adjacent square
⇒ $h_2(n)$ gives the shortest solution

- The relaxed problem adds edges to the state space
⇒ any optimal solution in the original problem is also a solution in the relaxed problem
⇒ the cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
- the derived heuristic is an exact cost for the relaxed problem
⇒ must obey the triangular inequality
⇒ consistent

Inferring Automatically Admissible Heuristics

Idea: If a problem definition is written down in a formal language, it is possible to construct relaxed problems automatically

Example

- 8-puzzle actions:

- a tile can move from square A to square B if
 - A is horizontally or vertically adjacent to B, and
 - B is blank

- we can generate three relaxed problems by removing one or both of the conditions

- (a) a tile can move from square A to square B if A is adjacent to B
 - (b) a tile can move from square A to square B if B is blank
 - (c) a tile can move from square A to square B

⇒ (a) corresponds to $h_2(n)$, (c) corresponds to $h_1(n)$,

The tool ABSolver can generate such heuristics automatically.

Learning Admissible Heuristics

- Another way to find an admissible heuristic is through **learning from experience**:
 - Experience = solving lots of 8-puzzles
 - An **inductive learning** algorithm can be used to predict costs for other states that arise during search